

Cortex AI — Merged Project Documents

Combined from PROMPTS.docx and AI-Project-Complete-Code.docx

PROMPTS.docx

Output: ████████ identity.py File Generate █████ ██████(CLAUDE AI) ROLE

You are a Senior Python AI Architect.

Build production-ready code for a Hackathon project called "Cortex AI".

Project Goal:

Create the Brain Module for an Autonomous AI Creator.

Task:

Create only the file brain/identity.py.

Requirements:

- Python 3.12
- Use OOP
- Use Dataclasses
- Use Type Hints
- Follow PEP8
- Use Google Style Docstrings
- Clean Architecture
- Production-ready code

Create:

1. WritingStyle Enum
2. Tone Enum
3. PersonaIdentity Dataclass
4. IdentityFactory Class

PersonaIdentity should contain:

- agent_id
- name
- domain
- writing_style
- tone
- audience
- interests
- editorial_rules
- forbidden_topics
- signature
- confidence_threshold
- posting_frequency_minutes
- created_at
- updated_at
- is_active

IdentityFactory should:

- Generate default persona
- Create persona from API input
- Validate persona
- Update persona
- Return dictionary

Validation:

- Domain required
- Confidence between 0 and 1
- Posting frequency > 0
- Audience cannot be empty
- Interests cannot be empty

Default Persona:

Name: Nova

Domain: AI Technology

Writing Style: Analytical

Tone: Professional

Audience:

Developers

Researchers

Startup Founders

Interests:

LLMs

AI Agents

Machine Learning

Open Source

Generative AI

Editorial Rules:

- Never publish clickbait
- Always cite sources
- Avoid duplicate posts
- Prefer technical depth

Forbidden Topics:

Politics

Religion

Celebrity Gossip

Sports

Output only identity.py code.

No explanation.

PERSONA.PY CODE

OUTPUT- ONLY CREATE THE FILE

ROLE

You are a Senior Python AI Architect.

You are building a production-ready Hackathon project called Cortex AI.

Previous file identity.py is already completed.

Now create only brain/persona.py.

The Persona module must use PersonaIdentity from identity.py.

Requirements:

- Python 3.12

- OOP
- Type Hints
- PEP8
- Google Style Docstrings
- Production Ready

Responsibilities:

Create a PersonaEngine class.

The PersonaEngine should:

- Load PersonaIdentity
- Maintain a consistent writing style
- Maintain a consistent tone
- Maintain audience preferences
- Check whether a topic matches persona interests
- Reject topics outside persona domain
- Generate writing instructions for the Publisher module
- Return persona profile
- Return writing guideline
- Support future extension

Methods:

- load_persona()
- get_profile()
- is_topic_relevant()
- get_writing_instruction()
- update_persona()
- export()

The engine should always ensure the AI behaves like the same person across every post.

Output only persona.py code.

No explanation.

brain\OPINION.PY -

ROLE

You are a Senior Python AI Architect building a production-ready Hackathon project called "Cortex AI".

Previous files identity.py and persona.py are already completed.

Now generate ONLY the file:

brain/opinion.py

Requirements:

- Python 3.12
- OOP
- Type Hints
- PEP8
- Google Style Docstrings
- Clean Architecture
- Production-ready code

Import PersonaEngine.

Create a class named OpinionEngine.

Responsibilities:

- Analyze discovered AI and technology topics.
- Decide whether the topic deserves publishing.
- Generate an editorial opinion instead of simply summarizing news.
- Reject weak, repetitive or low-value topics.
- Explain why a topic is accepted or rejected.
- Assign a confidence score (0-1).
- Maintain a consistent editorial philosophy.

Methods:

- __init__()
- analyze_topic()
- score_topic()
- evaluate_relevance()
- evaluate_novelty()
- generate_editorial_opinion()
- should_publish()
- get_publish_reason()
- export()

Evaluation Criteria:

- AI relevance
- Technical depth
- Industry impact
- Educational value
- Innovation
- Originality
- Duplicate possibility

Reject topics that are:

- Clickbait
- Celebrity gossip
- Politics
- Religion
- Sports
- Low technical value

Output for accepted topics should include:

- publish = True
- confidence score
- editorial opinion
- publishing reason

Output for rejected topics should include:

- publish = False
- rejection reason
- confidence score

Keep the code extensible for Memory, Discovery and Publisher modules.

Output only the complete opinion.py file.

Do not explain anything.

Brain\memory.py-

ROLE

You are a Senior Python AI Architect building a production-ready Hackathon project called "Cortex AI".

Previous files identity.py, persona.py and opinion.py are already completed.

Now generate ONLY the file:

brain/memory.py

Requirements:

- Python 3.12
- Object-Oriented Programming
- Type Hints
- PEP8
- Google Style Docstrings
- Clean Architecture
- Production-ready code

Create a class named MemoryEngine.

Responsibilities:

- Store every published post.
- Store rejected topics with rejection reasons.
- Maintain complete publishing history.
- Prevent duplicate content.
- Provide context for future posts.
- Retrieve recent posts.
- Search previous posts.
- Support future integration with database storage.

Methods:

- __init__()
- add_post()
- add_rejected_topic()
- get_post()
- get_all_posts()
- get_recent_posts()
- search_posts()
- topic_exists()
- get_memory_summary()
- clear_memory()
- export()

Each stored published post should contain:

- post_id
- topic
- generated_post
- editorial_opinion
- rationale
- source

- created_at
- confidence_score

Each rejected topic should contain:

- topic
- rejection_reason
- created_at

The Memory Engine should:

- Detect duplicate topics.
- Prevent repeated publishing.
- Keep posts ordered by latest first.
- Support future database integration.

Output only the complete memory.py file.

Do not explain anything.

Brain\knowledge.py---

ROLE

You are a Senior Python AI Architect building a production-ready Hackathon project called "Cortex AI".

Previous files identity.py, persona.py, opinion.py and memory.py are already completed.

Now generate ONLY the file:

brain/knowledge.py

Requirements:

- Python 3.12
- Object-Oriented Programming
- Type Hints
- PEP8
- Google Style Docstrings
- Clean Architecture
- Production-ready code

Create a class named KnowledgeEngine.

Responsibilities:

- Maintain the AI's technology knowledge domains.
- Decide whether a discovered topic belongs to the AI expertise.
- Categorize topics into knowledge areas.
- Assign a relevance score.
- Reject topics outside the knowledge base.
- Support future learning by adding new domains.
- Work with PersonaEngine and OpinionEngine.

Methods:

- __init__()
- load_domains()
- add_domain()
- remove_domain()
- get_domains()
- classify_topic()

- calculate_relevance()
- is_supported_topic()
- get_best_category()
- export()

Default Knowledge Domains:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Generative AI
- Large Language Models
- AI Agents
- Prompt Engineering
- Robotics
- Computer Vision
- NLP
- MLOps
- Open Source AI
- AI Security
- AI Ethics
- Cloud AI
- Python Development

Unsupported Topics:

- Politics
- Religion
- Celebrity News
- Sports
- Entertainment Gossip

For every topic return:

- category
- relevance_score
- supported (True/False)
- reason

Keep the code modular and ready for integration with Discovery, Memory and Publisher modules.

Output only the complete knowledge.py file.

Do not explain anything

Brain\duplicate.py-

ROLE

You are a Senior Python AI Architect building a production-ready Hackathon project called "Cortex AI".

Previous files identity.py, persona.py, opinion.py, memory.py and knowledge.py are already completed.

Now generate ONLY the file:

brain/duplicate.py

Requirements:

- Python 3.12
- Object-Oriented Programming

- Type Hints
- PEP8
- Google Style Docstrings
- Clean Architecture
- Production-ready code

Create a class named DuplicateDetector.

Responsibilities:

- Detect duplicate topics before publishing.
- Compare new topics with previously published topics.
- Compare similarity using keyword matching and text similarity.
- Prevent repeated publishing.
- Return similarity score.
- Explain why a topic is considered duplicate.
- Integrate with MemoryEngine.

Methods:

- __init__()
- normalize_text()
- calculate_similarity()
- compare_topics()
- is_duplicate()
- get_duplicate_reason()
- find_similar_posts()
- export()

Duplicate Detection Rules:

- Ignore uppercase/lowercase differences.
- Ignore punctuation.
- Ignore extra spaces.
- Support keyword-based comparison.
- Support future semantic similarity integration.

Output should include:

- is_duplicate (True/False)
- similarity_score
- matched_post_id
- matched_topic
- reason

The module must be modular and ready for integration with MemoryEngine, OpinionEngine and Publisher.

Output only the complete duplicate.py file.

Do not explain anything..

Brain\reflection.py-

ROLE

You are a Senior Python AI Architect building a production-ready Hackathon project called "Cortex AI".

Previous files (identity.py, persona.py, opinion.py, memory.py, knowledge.py and duplicate.py) are already completed.

Now generate ONLY the file:

brain/reflection.py

Requirements:

- Python 3.12
- Object-Oriented Programming
- Type Hints
- PEP8
- Google Style Docstrings
- Clean Architecture
- Production-ready code

Create a class named ReflectionEngine.

Responsibilities:

- Review every publishing decision.
- Analyze why a topic was accepted or rejected.
- Evaluate post quality before publishing.
- Generate a confidence report.
- Suggest improvements for future posts.
- Track publishing consistency.
- Work with PersonaEngine, OpinionEngine and MemoryEngine.

Methods:

- __init__()
- analyze_post()
- analyze_rejection()
- calculate_quality_score()
- calculate_confidence()
- generate_reflection()
- suggest_improvements()
- get_summary()
- export()

Reflection Report should include:

- quality_score
- confidence_score
- strengths
- weaknesses
- improvement_suggestions
- publishing_decision
- generated_at

Quality should be evaluated using:

- Topic relevance
- Technical depth
- Originality
- Editorial quality
- Persona consistency
- Duplicate risk
- Educational value
- Source credibility

Output should contain a structured reflection report that can later be shown through the API.

Keep the module modular and ready for integration with Publisher and API modules.

Output only the complete reflection.py file.

Do not explain anything

sources.py

Act as a senior Python backend architect building an autonomous AI technology persona for a hackathon.

We are implementing the Discovery Engine of an Autonomous AI Creator.

The system must independently discover current AI and technology topics without human prompts.

Create production-quality Python code for discovery/sources.py.

Requirements:

1. Create a clean source configuration system.
2. Support multiple live information sources, primarily RSS/Atom feeds so API keys are not required.
3. Include reliable AI/technology-oriented sources such as arXiv AI/ML feeds and technology/news RSS sources.
4. Do not hardcode individual news articles or fake topics.
5. Each source should have:
 - source name
 - URL
 - category
 - priority/reliability
 - enabled status
6. Use dataclasses or another clean typed structure.
7. Provide methods to:
 - return all enabled sources
 - return sources by category
 - validate source configuration
8. Keep the design extensible so more sources can be added later without modifying the fetcher.
9. Do not perform network requests in this file. Network fetching will be handled by fetcher.py.
10. Add proper type hints, docstrings, error handling, and clean Python architecture.
11. Do not create unnecessary dependencies.
12. Keep the implementation compatible with Python 3.12.
13. Do not create API endpoints in this file.
14. Do not create fake fallback news data.
15. Return only the complete code for sources.py followed by a short explanation of the architecture.

Fetcher.py

You are a senior Python backend engineer working on my Cortex AI Discovery Engine project.

The previous module `sources.py` is already completed. Now create the next module:

FILE: discovery/fetcher.py

IMPORTANT:

- This module must work directly with the existing `sources.py`.
- Import and use:
 - SourceConfig
 - SourceRegistry
 - SourceType
 - default_registry
- Do NOT modify sources.py.
- Do NOT create fake articles, fake topics, mock RSS data, or hardcoded news.
- Do NOT add a database layer here.
- Do NOT add Flask/FastAPI code here.
- Do NOT add AI/LLM processing here.

- Do NOT add filtering/scoring logic here. Those will be handled by filter.py and scorer.py.
- This module is ONLY responsible for fetching and parsing live RSS/Atom feeds.

REQUIREMENTS:

1. Create a clean `FeedFetcher` class.

2. It should fetch feeds from:

`SourceRegistry.get_enabled_sources()`

3. Support both:

- RSS
- Atom

4. Use reliable Python libraries such as:

- requests
- feedparser

5. Add configurable settings:

- timeout
- user-agent
- maximum articles per source

6. Handle network failures safely:

- timeout
- connection errors
- invalid HTTP responses
- malformed feeds
- parsing errors

One failed source must NOT stop the complete discovery process.

7. Parse each feed item and normalize it into a consistent dictionary structure containing:

- title
- url
- description
- published
- source_name
- source_url
- category
- priority
- tags

8. Remove empty or invalid article URLs.

9. Avoid duplicate articles.

Use the article URL as the primary unique identifier.

If URL is unavailable, use a reasonable fallback such as title + source.

10. Add methods such as:

`fetch_source(source: SourceConfig)`

`fetch_all(sources: Optional[List[SourceConfig]] = None)`

`fetch_enabled()`

11. `fetch_enabled()` must use the existing:

`default_registry.get_enabled_sources()`

12. Return clean Python dictionaries/lists so that later modules such as:

- filter.py
- scorer.py
- topic_queue.py

can consume the results easily.

13. Add proper type hints throughout the module.
14. Add useful logging using Python's `logging` module.
15. Do not print unnecessary output.
16. Add clear docstrings for the class and public methods.
17. Keep the implementation production-quality but simple and understandable for a student developer.
18. Make sure the module can be imported without automatically making network requests.
19. Add a small optional test/demo block under:

```
if __name__ == "__main__":
```

It should demonstrate how to create/use the fetcher, but do not run automatically when imported.

20. Before giving the final code, check that:

- imports are correct
- syntax is valid
- it matches the existing SourceConfig fields
- it does not modify sources.py
- it does not contain fake data
- it does not hardcode individual news articles

Return ONLY the complete `discovery/fetcher.py` code with no explanation.

Filter.py

Create the next module for the Cortex AI Discovery Engine:

File: discovery/filter.py

Purpose:

Build a clean, typed article filtering module that receives the normalized article dictionaries produced by fetcher.py and removes low-quality, irrelevant, duplicate, or invalid articles before they reach the scoring stage.

Requirements:

1. Do NOT fetch anything from the internet.
2. Do NOT use hardcoded articles or fake topics.
3. Use only the article data received from fetcher.py.
4. Keep this module independent from network fetching and AI/LLM processing.
5. Use Python type hints throughout.
6. Use dataclasses where appropriate.
7. Add clear docstrings and logging.
8. The module must work with the output format of fetcher.py.

Create:

- FilterConfig dataclass with sensible configurable values such as:
- min_title_length
- min_description_length
- max_title_length
- blocked_words
- required_keywords
- remove_duplicates

- require_url
- ArticleFilter class with methods:
 - validate_article(article)
 - is_duplicate(article)
 - is_relevant(article)
 - filter_article(article)
 - filter_articles(articles)

9. Validation should reject articles when:
 - title is missing or too short
 - URL is missing or invalid
 - article structure is invalid
10. Duplicate detection should use the article URL as the primary identifier.
11. Relevance filtering should support configurable keywords/tags but should NOT require a fixed hardcoded AI topic list.
12. Blocked words should be configurable through FilterConfig.
13. Preserve the original article dictionary fields.
14. Add a "filter_reason" or equivalent diagnostic mechanism only if it does not break downstream compatibility.
15. filter_articles() should:
 - process articles independently
 - remove invalid articles
 - remove duplicates
 - apply relevance filtering
 - return a clean List[Dict[str, Any]]
16. Handle unexpected malformed article data gracefully and log warnings instead of crashing the complete pipeline.
17. Add a small optional `if __name__ == "__main__":` demo that uses manually created sample dictionaries only for testing and does not affect imports.
18. Do not modify sources.py or fetcher.py.
19. Do not add external dependencies. Use Python standard library only.
20. Make the implementation production-quality and compatible with the existing Cortex AI Discovery Engine architecture.

Return ONLY the complete Python code for discovery/filter.py.

Score.py

"""

Cortex AI Discovery Engine - Article Filter Module

Module: filter.py

Purpose: Receives normalized article dictionaries from fetcher.py and filters out low-quality, invalid, duplicate, short, or off-topic articles prior to scoring and topic queuing.

"""

```
from dataclasses import dataclass, field
import logging
from typing import Any, Dict, List, Optional, Set
from urllib.parse import urlparse
```

```
# Configure logger
logger = logging.getLogger(__name__)
```

```
@dataclass
```

```
class FilterConfig:
```

```
    """Configuration settings for article filtering rules."""
```

```

minimum_title_length: int = 10
maximum_title_length: int = 300
minimum_description_length: int = 0
blocked_keywords: Set[str] = field(default_factory=set)
required_keywords: Set[str] = field(default_factory=set)
remove_duplicates: bool = True
case_sensitive_keywords: bool = False

def __post_init__(self) -> None:
    """Ensure keywords are standardized according to case sensitivity settings."""
    if not self.case_sensitive_keywords:
        self.blocked_keywords = {kw.strip().lower() for kw in self.blocked_keywords if kw and kw.strip()}
        self.required_keywords = {kw.strip().lower() for kw in self.required_keywords if kw and kw.strip()}
    else:
        self.blocked_keywords = {kw.strip() for kw in self.blocked_keywords if kw and kw.strip()}
        self.required_keywords = {kw.strip() for kw in self.required_keywords if kw and kw.strip()}

class ArticleFilter:
    """
    Production-ready filter pipeline for article dictionaries.
    Ensures data integrity, quality control, deduplication, and keyword filtering.
    """

    def __init__(self, config: Optional[FilterConfig] = None) -> None:
        """Initialize the filter with optional custom configuration."""
        self.config = config or FilterConfig()

    @staticmethod
    def _normalize_url(raw_url: str) -> str:
        """Normalize URL string for robust duplicate detection."""
        if not isinstance(raw_url, str):
            return ""
        url = raw_url.strip().rstrip("/")
        try:
            parsed = urlparse(url)
            # Remove default ports and force lowercase scheme & hostname
            scheme = parsed.scheme.lower()
            netloc = parsed.netloc.lower()
            path = parsed.path
            return f"{scheme}://{netloc}{path}"
        except Exception:
            return url.lower()

    def is_valid_article(self, article: Dict[str, Any]) -> bool:
        """
        Validates structural integrity, URL validity, title length, and description requirements.
        Never throws exceptions; returns False on invalid/malformed structures.
        """
        if not isinstance(article, dict):
            logger.debug("Filtered out article: Input is not a dictionary.")
            return False

        try:
            # 1. Validate Title

```

```

title = article.get("title")
if not isinstance(title, str) or not title.strip():
    logger.debug("Filtered out article: Title is missing or empty.")
    return False

clean_title = title.strip()
title_len = len(clean_title)
if title_len < self.config.minimum_title_length:
    logger.debug("Filtered out article: Title too short (%d chars): '%s'", title_len, clean_title)
    return False
if title_len > self.config.maximum_title_length:
    logger.debug("Filtered out article: Title too long (%d chars): '%s'", title_len, clean_title[:30])
    return False

# 2. Validate URL
url = article.get("url")
if not isinstance(url, str) or not url.strip():
    logger.debug("Filtered out article: URL is missing or empty.")
    return False

clean_url = url.strip()
parsed_url = urlparse(clean_url)
if not (parsed_url.scheme in ("http", "https") and parsed_url.netloc):
    logger.debug("Filtered out article: Invalid HTTP/HTTPS URL: '%s'", clean_url)
    return False

# 3. Validate Description
description = article.get("description") or ""
if not isinstance(description, str):
    description = str(description)

clean_desc = description.strip()
if self.config.minimum_description_length > 0:
    if len(clean_desc) < self.config.minimum_description_length:
        logger.debug("Filtered out article: Description too short (%d chars).", len(clean_desc))
        return False

return True

except Exception as err:
    logger.warning("Unexpected error during article validation: %s", err)
    return False

def is_duplicate(self, article: Dict[str, Any], seen_urls: Set[str]) -> bool:
    """
    Checks if an article's URL has already been processed.
    If remove_duplicates is enabled and URL is seen, returns True.
    Otherwise adds normalized URL to seen_urls and returns False.
    """
    if not self.config.remove_duplicates:
        return False

    if not isinstance(article, dict):
        return False

    raw_url = article.get("url", "")
    normalized_url = self._normalize_url(raw_url)

```



```

if not normalized_url:
    return False

if normalized_url in seen_urls:
    logger.debug("Duplicate detected for URL: %s", normalized_url)
    return True

seen_urls.add(normalized_url)
return False

def matches_keywords(self, article: Dict[str, Any]) -> bool:
    """
    Evaluates article against configured blocked and required keywords.
    Inspects title, description, and tags fields safely.
    """
    if not self.config.blocked_keywords and not self.config.required_keywords:
        return True

    if not isinstance(article, dict):
        return False

    try:
        # Extract text elements safely
        title = str(article.get("title") or "")
        description = str(article.get("description") or "")
        tags_list = article.get("tags") or []
        tags_text = " ".join(str(t) for t in tags_list) if isinstance(tags_list, (list, tuple, set)) else ""

        content = f"{title} {description} {tags_text}"
        if not self.config.case_sensitive_keywords:
            content = content.lower()

        # Check Blocked Keywords (Reject if ANY match)
        for blocked in self.config.blocked_keywords:
            if blocked in content:
                logger.debug("Filtered out article: Matched blocked keyword '%s'", blocked)
                return False

        # Check Required Keywords (Require AT LEAST ONE match if rule is configured)
        if self.config.required_keywords:
            matched_required = any(req in content for req in self.config.required_keywords)
            if not matched_required:
                logger.debug("Filtered out article: Failed required keywords check.")
                return False

        return True

    except Exception as err:
        logger.warning("Unexpected error during keyword evaluation: %s", err)
        return False

def filter_articles(self, articles: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
    Filters a list of normalized article dictionaries.

    Guarantees:
    - Input list and inner dictionaries are never mutated.

```

```

- Malformed or corrupt articles do not crash execution.
- Detailed statistics are logged upon completion.
"""

if not isinstance(articles, list):
    logger.error("filter_articles expects a list, received %s", type(articles))
    return []

filtered_articles: List[Dict[str, Any]] = []
seen_urls: Set[str] = set()

stats = {
    "total_received": len(articles),
    "passed": 0,
    "invalid": 0,
    "duplicates": 0,
    "keyword_mismatch": 0,
    "processing_errors": 0,
}

for article in articles:
    try:
        # Basic validation
        if not self.is_valid_article(article):
            stats["invalid"] += 1
            continue

        # Duplicate detection
        if self.is_duplicate(article, seen_urls):
            stats["duplicates"] += 1
            continue

        # Keyword filtering
        if not self.matches_keywords(article):
            stats["keyword_mismatch"] += 1
            continue

        # Passed all filters
        filtered_articles.append(article)
        stats["passed"] += 1

    except Exception as err:
        stats["processing_errors"] += 1
        logger.error("Uncaught exception filtering article: %s", err, exc_info=True)

    logger.info(
        "Article filtering complete: Total=%d | Passed=%d | Invalid=%d | Duplicates=%d | Keyword Filtered=%d | Errors=%d",
        stats["total_received"],
        stats["passed"],
        stats["invalid"],
        stats["duplicates"],
        stats["keyword_mismatch"],
        stats["processing_errors"],
    )

    return filtered_articles

def filter_by_category(self, articles: List[Dict[str, Any]], category: str) -> List[Dict[str, Any]]:

```

```

"""
Filters input articles matching the specified category (case-insensitive).
Returns a new list without modifying original inputs.
"""

if not isinstance(articles, list) or not isinstance(category, str):
    return []

target_category = category.strip().lower()
result: List[Dict[str, Any]] = []

for article in articles:
    if isinstance(article, dict):
        art_category = str(article.get("category") or "").strip().lower()
        if art_category == target_category:
            result.append(article)

return result

def filter_by_source(self, articles: List[Dict[str, Any]], source_name: str) -> List[Dict[str, Any]]:
    """
    Filters input articles matching the specified source_name (case-insensitive).
    Returns a new list without modifying original inputs.
    """

    if not isinstance(articles, list) or not isinstance(source_name, str):
        return []

    target_source = source_name.strip().lower()
    result: List[Dict[str, Any]] = []

    for article in articles:
        if isinstance(article, dict):
            art_source = str(article.get("source_name") or "").strip().lower()
            if art_source == target_source:
                result.append(article)

    return result

# -----
# Demonstration / Local Verification Block
# -----

if __name__ == "__main__":
    # Setup standard console logging for demonstration
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
    )

    logger.info("Starting local demonstration for filter.py ...")

    # Sample test dataset mimicking fetcher.py normalized dictionaries
    sample_articles: List[Dict[str, Any]] = [
        {
            "title": "Breakthrough in Generative AI Architecture Unveiled",
            "url": "https://tech-news.org/articles/gen-ai-breakthrough",
            "description": "Researchers introduce a highly scalable neural transformer design.",
            "published": "2026-08-08T10:00:00Z",

```

```
"source_name": "TechNews",
"source_url": "https://tech-news.org",
"category": "Artificial Intelligence",
"priority": 1,
"tags": ["AI", "Machine Learning", "Research"],
},
{
# Duplicate URL (normalized match with article 1)
"title": "Breakthrough in Generative AI Architecture Unveiled!",
"url": "https://tech-news.org/articles/gen-ai-breakthrough/",
"description": "Researchers introduce a highly scalable neural transformer design.",
"published": "2026-08-08T10:05:00Z",
"source_name": "TechNews",
"source_url": "https://tech-news.org",
"category": "Artificial Intelligence",
"priority": 1,
"tags": ["AI"],
},
{
# Invalid: Title too short
"title": "Short",
"url": "https://example.com/short-title",
"description": "Valid description length for testing purposes.",
"published": "2026-08-08T11:00:00Z",
"source_name": "ExampleBlog",
"source_url": "https://example.com",
"category": "General",
"priority": 2,
"tags": [],
},
{
# Invalid: Missing scheme/bad URL
"title": "Invalid URL Article Format",
"url": "not-a-valid-url-string",
"description": "This article has a completely broken URL format.",
"published": "2026-08-08T12:00:00Z",
"source_name": "BadSource",
"source_url": "http://bad.source",
"category": "Engineering",
"priority": 3,
"tags": ["Bug"],
},
{
# Contains Blocked Keyword ("crypto")
"title": "New Crypto Scam targeting Web3 Users Exposed",
"url": "https://security-today.com/crypto-scam-alert",
"description": "Security analysts identify phishing campaigns targeting wallets.",
"published": "2026-08-08T13:00:00Z",
"source_name": "SecurityToday",
"source_url": "https://security-today.com",
"category": "Security",
```

```

"priority": 1,
"tags": ["Security", "Crypto"],
},
{
# Valid AI Article
"title": "Next-Gen Quantum Computing Chips Enter Production",
"url": "https://hardware-weekly.com/quantum-chip-production",
"description": "Fab units report successful yield for 100-qubit processors.",
"published": "2026-08-08T14:00:00Z",
"source_name": "HardwareWeekly",
"source_url": "https://hardware-weekly.com",
"category": "Hardware",
"priority": 2,
"tags": ["Quantum", "Hardware"],
},
# Malformed item in batch
None, # type: ignore
"Invalid non-dict item", # type: ignore
]

# Instantiate Filter Configuration
config = FilterConfig(
    minimum_title_length=10,
    minimum_description_length=15,
    blocked_keywords={"crypto", "casino", "sponsored"},
    required_keywords=set(), # Open filter
    remove_duplicates=True,
)

filter_engine = ArticleFilter(config=config)

# Execute main filter pipeline
print("\n--- Running Main Filter Pipeline ---")
clean_articles = filter_engine.filter_articles(sample_articles)

print(f"\nRemaining Articles ({len(clean_articles)}):")
for idx, art in enumerate(clean_articles, 1):
    print(f"{idx}. [{art['source_name']}] {art['title']} ({art['url']})")

# Demonstrate Category Sub-filtering
print("\n--- Filtering by Category: 'Artificial Intelligence' ---")
ai_articles = filter_engine.filter_by_category(clean_articles, "Artificial Intelligence")
print(f"Found {len(ai_articles)} AI article(s):")
for art in ai_articles:
    print(f"- {art['title']}")

# Demonstrate Source Sub-filtering
print("\n--- Filtering by Source: 'HardwareWeekly' ---")
hw_articles = filter_engine.filter_by_source(clean_articles, "HardwareWeekly")
print(f"Found {len(hw_articles)} HardwareWeekly article(s):")
for art in hw_articles:
    print(f"- {art['title']}")

pipeline.py-
```

Create a production-ready `pipeline.py` module for the Cortex AI Discovery Engine.

IMPORTANT:

The following modules already exist and must be imported and used. Do NOT recreate or replace them:

- `discovery.sources`
- `discovery.fetcher`
- `discovery.filter`
- `discovery.scorer`
- `discovery.topic_queue`

The pipeline flow must be:

SourceRegistry

↓

FeedFetcher

↓

ArticleFilter

↓

ArticleScorer

↓

TopicQueue

Create a `DiscoveryPipeline` class that coordinates the complete discovery workflow.

Requirements:

1. Create a `PipelineConfig` dataclass with sensible configuration options for:

- `fetcher` configuration
- `filter` configuration
- `scorer` configuration
- `topic queue` configuration

2. Initialize and manage:

- `FeedFetcher`
- `ArticleFilter`
- `ArticleScorer`
- `TopicQueue`

3. Implement these methods:

`fetch_articles()`

- Fetch articles from enabled sources using `FeedFetcher`.
- Return the raw normalized articles.

`filter_articles(articles)`

- Pass fetched articles through `ArticleFilter`.
- Return only valid/clean articles.

`score_articles(articles)`

- Score and rank articles using `ArticleScorer`.
- Return scored and ranked articles.

`queue_articles(articles)`

- Add scored articles to `TopicQueue`.
- Return the number of successfully queued articles.

`run()`

- Execute the complete pipeline in this order:

`fetch` → `filter` → `score/rank` → `queue`

- Return useful pipeline results/statistics.

``get_topics()``

- Return all currently queued topics.

``get_top_topic()``

- Return the highest-ranked queued topic or None.

``clear_queue()``

- Clear the TopicQueue.

4. Add pipeline statistics such as:

- `fetches_count`
- `filtered_count`
- `scored_count`
- `queued_count`

5. Use proper Python type hints.

6. Add detailed logging using Python's ``logging`` module.

7. Add defensive error handling:

- A failed feed/source must not crash the pipeline.
- Malformed articles must not crash the pipeline.
- Handle unexpected errors gracefully.

8. Do NOT mutate article dictionaries received from previous modules.

9. Use the existing classes exactly as they are:

- `FeedFetcher`
- `FetcherConfig`
- `ArticleFilter`
- `FilterConfig`
- `ArticleScorer`
- `ScoreConfig`
- `TopicQueue`
- `TopicQueueConfig`
- `SourceRegistry`
- `default_registry`

10. Add a clean ``close()`` method that closes `FeedFetcher` resources.

11. Support context-manager usage:

``with DiscoveryPipeline() as pipeline:``

12. Add a ``__repr__()`` method.

13. Include a ``if __name__ == "__main__":`` demonstration block that:

- creates the pipeline
- runs it
- prints fetched, filtered, scored and queued counts
- prints the top topic
- closes resources properly

14. Keep the implementation modular, readable, production-ready, and consistent with the coding style of the existing Cortex AI Discovery Engine modules.

Return ONLY the complete Python code for ``pipeline.py``.

AI-CONTENT INTELLIGENCE-

You are a senior Python architect and AI systems engineer working on a production-grade hackathon project called "Cortex AI Discovery Engine".

PROJECT CONTEXT:

Module 1 (Discovery Engine) is already completed and must NOT be modified unnecessarily.

Module 1 currently provides:

- discovery/sources.py
- discovery/fetcher.py
- discovery/filter.py
- discovery/scorer.py
- discovery/topic_queue.py
- discovery/pipeline.py
- discovery/__init__.py

The pipeline produces clean, scored article dictionaries containing fields such as:

title, url, description, published, source_name, source_url, category, priority, tags, score, score_breakdown.

OBJECTIVE:

Build Module 2: "AI Content Intelligence".

PHASE 1 NAME:

"AI Processing Foundation / Article Understanding Layer"

IMPORTANT:

Do NOT build the complete AI intelligence system yet.

This phase is ONLY the foundation that later phases will use for summarization, topic extraction, categorization, entity extraction, keyword extraction, and insight generation.

CREATE:

A new package:

ai_intelligence/

Inside it create the minimum necessary files for Phase 1 only.

REQUIREMENTS:

1. AI PROVIDER ABSTRACTION

Create a clean provider interface/abstraction so the rest of the application does NOT depend directly on one AI vendor.

The design should allow providers such as:

- OpenAI
- Gemini
- Anthropic
- Local models

to be added later without rewriting the rest of the application.

2. CONFIGURATION

Create a configuration structure for AI processing containing sensible settings such as:

- provider name
- model name
- API timeout
- maximum input characters/tokens
- temperature where applicable
- retry count
- enable/disable AI processing

Do not hard-code API keys.

API keys must be loaded from environment variables.

3. ARTICLE INPUT NORMALIZATION

Create a component that safely converts the normalized article dictionary from Module 1 into an AI-ready text representation.

It should:

- safely handle missing title
- safely handle missing description
- safely handle tags
- avoid crashing on malformed input
- enforce configurable input length limits
- preserve the original article dictionary

4. AI RESPONSE CONTRACT

Define a structured response model/contract for future AI analysis.

It should be designed to eventually support fields such as:

- summary
- topics
- category
- keywords
- entities
- importance/relevance explanation
- confidence

However, Phase 1 should NOT implement all these AI features.

5. PROVIDER INTERFACE

Create an abstract/base provider with a method similar to:

```
analyze_article(...)
```

The method should have a clear contract and type hints.

Provide at least one safe placeholder/mock implementation for local testing so Phase 1 can be tested WITHOUT a real API key.

Do NOT make real external AI API calls in the local demonstration.

6. ERROR HANDLING

The AI layer must gracefully handle:

- missing API key
- provider unavailable
- timeout
- malformed response
- invalid article input
- unexpected exceptions

Do not allow one failed article to crash the complete processing pipeline.

7. LOGGING

Use Python logging properly.

Include useful logs for:

- provider initialization
- article preparation
- processing attempts
- failures
- fallback/mock processing

Do not log API keys or sensitive credentials.

8. IMMUTABILITY

Never mutate the original article dictionary received from Module 1.

Return new dictionaries/objects.

9. TYPE SAFETY

Use:

- dataclasses where appropriate
- typing
- Optional
- Dict/List or modern Python type syntax consistently

Code should be clean and production-oriented.

10. TEST/DEMO

Include a small local demonstration or testable example showing:

Module 1 article dictionary

↓

article preparation

↓

AI provider abstraction

↓

mock provider

↓

structured AI result

The demonstration must work without any API key and without making network requests.

11. DEPENDENCIES

Keep Phase 1 lightweight.

Do not add unnecessary third-party libraries.

Prefer Python standard library wherever possible.

12. ARCHITECTURE

Keep responsibilities separated.

For example, conceptually:

```
ai_intelligence/  
  __init__.py  
  config.py  
  models.py  
  article_preprocessor.py  
  providers/  
    __init__.py  
    base.py  
    mock.py
```

You may adjust the exact filenames if you have a clearly better architecture, but do NOT over-engineer the project.

13. INTEGRATION

Do NOT modify discovery/pipeline.py yet.

Do NOT connect this module to the production discovery pipeline in this phase.

Only make the AI foundation importable and ready for integration in a later phase.

14. CODE QUALITY

Write complete runnable Python code, not pseudocode.

Include:

- module docstrings
- class/function docstrings
- type hints
- defensive validation
- meaningful names
- clean architecture

Avoid:

- unnecessary comments
- placeholder TODOs that break execution
- fake API keys
- hard-coded secrets
- unnecessary frameworks

15. FINAL OUTPUT

After generating the code, provide:

- Final folder structure
- Complete code for every created file
- Dependencies, if any
- Exact command to run the local demonstration
- What the demonstration should output
- Brief explanation of how Phase 1 will connect to later AI Intelligence phases

REMEMBER:

This is Phase 1 only.

Do NOT implement:

- article summarization using a real LLM
- topic extraction using a real LLM
- entity extraction
- embeddings
- vector database
- RAG
- semantic search
- trend detection
- autonomous agents

Those will be implemented in later phases.

The goal of this phase is a robust, extensible AI processing foundation that can plug into the existing Cortex AI Discovery Engine later.

Ai_intelligence/providers/openai_providers prompt-

ROLE:

Act as a world-class AI systems architect and senior Python engineer specializing in production-grade hackathon systems.

PROJECT:

Cortex AI Discovery Engine.

CONTEXT:

This is Module 2 of the project.

Module 2 Phase 1 is already implemented and MUST NOT be broken or redesigned.

Existing architecture:

```
ai_intelligence/  
■■■ __init__.py  
■■■ config.py  
■■■ models.py  
■■■ article_preprocessor.py  
■■■ demo.py  
■■■ providers/  
■■■ __init__.py  
■■■ base.py  
■■■ mock.py
```

Existing classes/contracts:

- AIConfig
- PreparedArticleInput
- AnalysisResult
- ArticlePreprocessor
- BaseAIProvider
- MockAIProvider

BaseAIProvider exposes:

analyze_article(article_input: PreparedArticleInput) -> AnalysisResult

and requires subclasses to implement:

_execute_analysis(article_input: PreparedArticleInput) -> AnalysisResult

TASK:

Create ONLY the file:

ai_intelligence/providers/openai_provider.py

Implement a production-oriented OpenAI provider named:

OpenAIProvider(BaseAIProvider)

REQUIREMENTS:

1. Use the existing BaseAIProvider interface exactly.
2. Do NOT modify any existing file.
3. Use AIConfig from ai_intelligence.config.
4. Use PreparedArticleInput and AnalysisResult from ai_intelligence.models.
5. Read the API key from AIConfig.api_key.
6. Never hard-code API keys.
7. Use the official OpenAI Python SDK.
8. Keep the implementation vendor-specific only inside this provider.
9. Do not expose OpenAI-specific response structures outside this class.
10. Return AnalysisResult in every successful or handled failure case.
11. Never crash the complete discovery pipeline because of an AI API failure.
12. Handle:
 - missing API key
 - API timeout
 - API/network failure
 - malformed AI response
 - unexpected exceptions

13. Respect `AIConfig.api_timeout`.
14. Respect `AIConfig.retry_count`.
15. Respect `AIConfig.max_input_chars`.
16. Use the prepared article's `formatted_prompt_text` as the model input.
17. Ask the model to return STRICT JSON only.
18. Expected JSON structure:

```
{
  "summary": "...",
  "topics": [...],
  "category": "...",
  "keywords": [...],
  "entities": [...],
  "importance_explanation": "...",
  "confidence": 0.0
}
```

19. Validate the returned JSON before constructing `AnalysisResult`.
20. Confidence must be safely converted to a float between 0.0 and 1.0.
21. Missing fields should receive safe defaults.
22. Set:
`provider_name = self.config.provider_name`
`model_name = self.config.model_name`
23. Include useful logging but NEVER log the API key or sensitive credentials.
24. Keep the implementation clean, typed, readable, and production-oriented.
25. Use defensive programming.
26. Do not create unnecessary abstractions.
27. Add a small helper method if necessary for parsing/validation.
28. Do not add a demo block to this file.
29. Return `status="success"` for valid successful analysis.
30. Return `status="fallback"` when the provider cannot execute because the API key is unavailable.
31. Return `status="error"` for API or parsing failures.
32. Preserve the `article_id` from `PreparedArticleInput`.

IMPORTANT:

The generated code must integrate directly with the existing `BaseAIProvider` and `AnalysisResult` contracts without changing them.

OUTPUT:

Return ONLY the complete Python code for `openai_provider.py`.

Do not explain anything outside the code.

`ai_intelligence\analysis_engine.py` prompt-

You are a senior AI systems architect and hackathon-winning Python engineer.

I am building a modular AI-powered discovery engine called "Cortex AI Discovery Engine".

IMPORTANT:

Do NOT modify any existing files or architecture.

Do NOT rewrite Module 1.

Do NOT change the existing Phase 1 contracts.

The new code must integrate cleanly with the existing architecture.

Current Module 2 Phase 1 already contains:

```
ai_intelligence/
config.py
```

```
models.py
article_preprocessor.py
demo.py
__init__.py
providers/
base.py
mock.py
__init__.py
```

Existing components:

```
AIConfig
PreparedArticleInput
AnalysisResult
ArticlePreprocessor
BaseAIProvider
MockAIProvider
```

BaseAIProvider already exposes:

```
analyze_article(article_input: PreparedArticleInput) -> AnalysisResult
```

The MockAIProvider already returns a structured AnalysisResult.

TASK:

Create ONE new production-quality Python module:

```
ai_intelligence/analysis_engine.py
```

Create a class:

```
AIAnalysisEngine
```

The engine must act as the orchestration layer between the article preprocessor and AI provider.

Required responsibilities:

1. Accept an AIConfig and a BaseAIProvider through dependency injection.
2. Accept an ArticlePreprocessor through dependency injection when appropriate.
3. Provide:

```
analyze_article(article: Dict[str, Any]) -> AnalysisResult
```

4. The method must:

- validate the incoming article safely
- preprocess it using ArticlePreprocessor
- send the PreparedArticleInput to BaseAIProvider.analyze_article()
- return the resulting AnalysisResult
- never allow unexpected exceptions to crash the pipeline

5. Provide batch processing:

```
analyze_articles(articles: List[Dict[str, Any]]) -> List[AnalysisResult]
```

6. Batch processing must:

- safely process malformed items
- preserve article processing order
- never mutate original article dictionaries
- return a structured AnalysisResult for failures instead of crashing

7. Add useful logging:

- article preparation
- provider execution
- success/failure
- batch statistics

8. Add defensive validation for:

- non-dictionary article
- missing title
- missing URL
- malformed preprocessor result
- provider returning an invalid result

9. If provider returns something other than AnalysisResult, convert it into a safe error AnalysisResult.

10. Add a helper method if useful:

```
analyze_prepared_article(
    article_input: PreparedArticleInput
) -> AnalysisResult
```

11. Keep the engine provider-agnostic.

Do NOT import OpenAI, Gemini, Anthropic, requests, or any external AI SDK.

12. Do NOT make network calls.

13. Use only the existing project abstractions and Python standard library.

14. Add type hints everywhere.

15. Add docstrings for the class and public methods.

16. Make the implementation production-quality, clean, modular, defensive, and easy to extend.

17. Add a local demonstration block under:

```
if __name__ == "__main__":
```

The demo must:

- create a sample normalized/scored article
- create AIConfig with mock provider
- create ArticlePreprocessor
- create MockAIProvider
- create AIAnalysisEngine
- analyze the article
- print the AnalysisResult as JSON
- demonstrate batch processing with at least one malformed item
- verify that the original article was not mutated

18. IMPORTANT:

Do not create additional files.

Do not modify __init__.py yet.

Do not modify demo.py yet.

Only output the complete code for:

```
ai_intelligence/analysis_engine.py
```

Before giving the code, briefly verify that your implementation is compatible with the existing Phase 1 classes described above.

Publisher\engine.py prompt-

Role:

Act as a senior Python backend engineer helping me build the Publisher Engine for my hackathon project.

Project:

Cortex AI - Autonomous AI Creator.

Problem:

The agent must autonomously discover AI/technology topics, make editorial decisions, generate consistent persona-based posts, remember previously published content, and publish new posts over time without additional human prompts.

Important:

Do NOT redesign or rewrite the existing project.

Do NOT create duplicate modules.

Do NOT modify the existing Brain, Discovery, or AI Intelligence modules unless absolutely necessary for integration.

First inspect the existing repository structure and existing code/interfaces.

Existing modules include:

brain/

- persona.py
- identity.py
- knowledge.py
- memory.py
- opinion.py
- duplicate.py
- reflection.py

discovery/

- sources.py
- filter.py
- scorer.py
- pipeline.py
- topic_queue.py
- sources/

ai_intelligence/

- analysis_engine.py
- article_preprocessor.py
- config.py
- models.py
- providers/
- demo.py

api/

- routes.py

main.py already contains a Flask application and registers the API blueprint.

Your task:

Implement the Publisher Module inside:

publisher/

The Publisher Module must act as the bridge between:

Discovery → Editorial Brain → Content Generation → Memory → API Feed.

Requirements:

1. Inspect existing modules first.

- Understand their public classes and methods.
- Reuse existing interfaces.
- Do not duplicate functionality already implemented in brain/ or discovery/.

2. Create a clean Publisher architecture.

Suggested responsibilities:

- publisher engine/orchestrator
- post generation
- publishing scheduler
- published-post storage/integration
- publishing result/model if required

3. The Publisher Engine should:

- receive a discovered topic/article candidate
- use the existing editorial/opinion logic
- reject topics that fail editorial criteria
- generate a persona-consistent post
- attach publishing rationale
- attach source URLs
- create a unique post ID
- create an ISO-8601 UTC createdAt timestamp
- send/store the post through the existing API/feed mechanism

4. Every published post MUST contain:

- id
- createdAt
- text
- rationale
- sources

5. The rationale must explain:

- why the topic was selected
- why it is relevant now
- why it was selected instead of weaker candidates

6. Autonomous publishing:

- Publishing must happen over time.
- Do NOT generate the complete feed immediately during initialization.
- Design the publisher so it can run in a background thread/scheduler after agent initialization.
- It should periodically check the discovery pipeline and publish eligible topics.
- Avoid publishing the same topic repeatedly.
- Previously published posts must remain available.

7. Since real social media integration is NOT required:

- use simulated/in-memory publishing if appropriate
- keep the architecture ready for future real-platform integration
- do not add unnecessary APIs

8. Safety against duplicate publishing:

- integrate with existing brain/duplicate.py and/or memory.py
- do not implement a second unrelated duplicate detector

9. Persona consistency:

- use the existing PersonaEngine / PersonaIdentity
- use its writing instructions

- generated posts should sound like the configured AI/technology persona
- keep content focused on AI and technology

10. Error handling:

- publisher should not crash the entire application because one topic fails
- log failures
- continue processing future topics

11. Thread safety:

- publishing may happen in a background thread
- use appropriate locking where shared in-memory state is accessed

12. Integration:

- make the Publisher easy to connect to:

POST /api/agent/init

GET /api/agent/feed?agentId=<id>

13. Do not add real social media APIs.

14. Do not add unnecessary dependencies.

15. Keep the implementation compatible with Python 3.12 and the existing Flask application.

Before writing code:

- inspect the existing repository
- identify the exact interfaces/classes that should be reused
- explain briefly which existing modules Publisher will integrate with
- identify any missing interface that must be added

Then implement only the Publisher Module and the minimum integration required.

After implementation:

- run syntax/compile checks
- run the existing pytest suite
- report exactly what files were created/modified
- report any integration issues
- do not claim tests passed unless they actually passed.

Code quality:

- clean Python
- type hints
- docstrings
- logging
- no unnecessary duplication
- maintainable architecture
- hackathon-demo friendly

Publisher\scheduler.py -

I am building a hackathon project called Cortex AI Autonomous Creator.

Problem requirement:

After POST /api/agent/init is called exactly once, the AI agent must operate autonomously for approximately 48 hours without receiving any further instructions or API calls. It must periodically discover AI/technology topics, apply editorial judgment, publish approved posts, and make new posts available through the feed endpoint.

Create ONLY the file:

publisher/scheduler.py

Do NOT modify any existing files.

Use the existing PublisherEngine from:

`publisher/engine.py`

The scheduler should:

1. Run as a background autonomous publishing loop after an agent is initialized.
2. Accept an agent/persona context and callbacks/dependencies for:
 - topic discovery
 - editorial analysis
 - publishing
3. Periodically execute:
Discovery → Editorial Judgment → Publisher
4. Never publish rejected topics.
5. Avoid publishing the same topic repeatedly.
6. Continue running after initialization without requiring another API request.
7. Use configurable publishing intervals rather than hard-coded long delays.
8. Support a small configurable delay for development/testing.
9. Run safely in a daemon/background thread so Flask/API remains responsive.
10. Prevent multiple scheduler loops from accidentally starting for the same agent.
11. Provide a clean stop mechanism for tests/shutdown.
12. Handle exceptions inside the autonomous loop so one failed cycle does not terminate the scheduler.
13. Log important lifecycle events:
 - scheduler started
 - discovery cycle started
 - topic rejected
 - topic published
 - cycle failed
 - scheduler stopped
14. Do not call real social media APIs.
15. Keep the implementation compatible with Python 3.12.
16. Keep the design modular so it can later be connected to the existing discovery and brain modules.

Expose a clear class, preferably:

`AutonomousPublisherScheduler`

with methods similar to:

```
start()
stop()
is_running()
```

and an internal publishing cycle.

The implementation must be production-structured but simple enough for a hackathon.

Important:

- Do not create fake API endpoints.
- Do not rewrite `PublisherEngine`.
- Do not modify `main.py`, `api/routes.py`, `discovery/`, `brain/`, or any other file.
- Add type hints and docstrings.
- Do not add unnecessary dependencies.
- Make it easy to unit test with a very small interval.

After generating the file, explain briefly what was created and how another module can start the scheduler.

Frontend prompts—

ROLE:

You are a world-class frontend engineer, UI/UX designer, creative technologist, and hackathon-winning product designer. You specialize in building highly polished, innovative, futuristic web interfaces using HTML5, CSS3, JavaScript, Bootstrap when useful, and modern browser APIs.

PROJECT CONTEXT:

We are building "Cortex AI Autonomous Creator" for a hackathon.

The backend is ALREADY WORKING in Flask.

Backend API:

GET <http://127.0.0.1:5000/api/agent/feed?agentId=cortex-main-agent>

The API returns:

```
{
  "posts": [
    {
      "id": "p-...",
      "createdAt": "...",
      "text": "...",
      "rationale": "...",
      "sources": ["..."]
    }
  ]
}
```

CURRENT AGENT:

Agent ID: cortex-main-agent

Persona name: Nova

Domain: AI Technology

IMPORTANT:

Do NOT rebuild or modify the backend.

Do NOT create fake/mock posts.

The frontend must fetch the real posts from the Flask API.

GOAL:

Create an extremely impressive hackathon frontend/dashboard for Cortex AI Autonomous Creator.

The interface should make judges immediately understand that this is an autonomous AI content discovery and publishing system.

DESIGN DIRECTION:

Create a futuristic "AI command center" / "autonomous intelligence dashboard".

Visual style:

- premium futuristic SaaS
- dark cinematic interface
- glassmorphism
- subtle neon cyan/blue/purple accents
- elegant gradients
- soft glowing borders
- subtle animated background
- modern typography
- excellent spacing
- professional, NOT childish
- visually impressive but still usable
- responsive on laptop, tablet and mobile

Do NOT make it look like a generic admin dashboard.

PAGE STRUCTURE:

1. TOP NAVIGATION

Create a premium navigation bar containing:

- Cortex logo/icon
- "CORTEX"
- "Autonomous Creator"
- Live status indicator
- Agent ID
- refresh button
- theme/control button if useful

Show:

"● SYSTEM ONLINE"

2. HERO / COMMAND CENTER

Large heading:

"CORTEX"

"Autonomous Intelligence Feed"

Subtitle:

"Discover. Evaluate. Publish. Automatically."

Add a visually impressive AI status/orbit visualization showing:

DISCOVER → ANALYZE → SCORE → PUBLISH

Animate the stages subtly.

Show live metrics:

- Articles discovered
- Articles evaluated
- Topics published
- Agent status

3. AGENT PROFILE CARD

Create a beautiful glass card for:

NOVA

AI Technology

Agent ID:

cortex-main-agent

Status:

ACTIVE

Show a small animated AI avatar/orb representing Nova.

4. LIVE ACTIVITY / PIPELINE

Create a horizontal or vertical autonomous pipeline:

SOURCE

↓

DISCOVERY

↓

FILTER

↓

SCORING

↓

EDITORIAL

↓

PUBLISH

Each stage should have:

- icon
- short description
- status
- subtle animation

Highlight completed stages with a glowing state.

5. MAIN CONTENT: AI PUBLISHED FEED

This is the most important section.

Heading:

"AI PUBLISHED INTELLIGENCE"

Fetch real data using JavaScript:

```
fetch(
  "http://127.0.0.1:5000/api/agent/feed?agentId=cortex-main-agent"
)
```

Render every post dynamically.

Each post should appear as a premium intelligence card.

Card must display:

- topic title
- generated editorial text
- rationale
- timestamp
- source link
- relevance score if available
- category if available

Use different visual indicators for:

HIGH relevance

MEDIUM relevance

LOW relevance

Do NOT hardcode the posts.

6. INTELLIGENCE CARD DESIGN

Each card should look like an AI-generated intelligence report.

Example layout:

[AI ANALYSIS]

ChatGPT starts blocking direct requests to copy an author's style

Interesting development in Technology...

WHY CORTEX SELECTED THIS

Selected automatically by Cortex AI Discovery Engine...

SOURCE

Read original →

Include:

- animated hover effect
- glowing left border
- source icon
- timestamp
- small AI badge

7. LIVE DATA BEHAVIOUR

The frontend should:

- fetch data when page loads
- show loading skeleton while fetching
- handle API errors gracefully
- show "No intelligence published yet" if feed is empty
- provide refresh button
- optionally auto-refresh every 30 seconds
- never crash if one field is missing

8. SYSTEM STATUS PANEL

Show:

SYSTEM STATUS

- API ONLINE
- DISCOVERY ENGINE ACTIVE
- PUBLISHER ACTIVE
- AGENT ACTIVE

Use JavaScript to check:

<http://127.0.0.1:5000/>

If API responds successfully, show ONLINE.

Otherwise show OFFLINE.

9. ANALYTICS SECTION

Create small futuristic charts/cards for:

- Topics discovered
- Topics published
- Average relevance score
- Publishing activity

Use CSS/HTML/vanilla JS only unless a chart library is absolutely necessary.

10. "HOW CORTEX THINKS" SECTION

Create an attractive visual explanation:

01 DISCOVER

Cortex scans configured sources.

02 FILTER

Invalid and duplicate topics are removed.

03 SCORE

Topics are evaluated for relevance.

04 EDIT

The AI creates an editorial perspective.

05 PUBLISH

Approved topics are automatically published.

Make this section visually impressive with connecting lines and animated nodes.

11. FOOTER

"CORTEX AI AUTONOMOUS CREATOR"

"Built for Hackathon 2026"

TECHNICAL REQUIREMENTS:

Use:

- HTML5
- CSS3
- Vanilla JavaScript
- Bootstrap only if useful

Create these exact files:

```
frontend/  
index.html  
style.css  
script.js
```

IMPORTANT:

Do not put CSS or JavaScript inside index.html unless absolutely necessary.

The frontend must work with Flask running on:

`http://127.0.0.1:5000`

Make API calls using JavaScript `fetch()`.

Because frontend and Flask may run on different origins, make the frontend compatible with CORS or explain exactly where CORS needs to be enabled. Prefer serving the frontend through Flask if possible.

QUALITY REQUIREMENTS:

- clean semantic HTML
- modular JavaScript
- readable code
- responsive design
- accessible buttons
- smooth animations
- no unnecessary libraries
- no fake data
- no lorem ipsum
- no broken links
- no console errors
- no horizontal scrolling
- polished loading states
- polished error states
- empty-state UI
- excellent mobile layout

HACKATHON JUDGE IMPACT:

The first viewport must immediately communicate:

"Cortex is an autonomous AI system that discovers technology/news topics, evaluates them, and publishes editorial intelligence automatically."

The interface should feel like a real product, not a college project.

Add subtle details that make the project feel premium:

- animated AI pulse
- glowing status indicators
- glass cards
- smooth card reveal animations
- intelligent empty/loading states
- dynamic counters
- live API status
- source links
- elegant hover interactions

DO NOT overdo animations.

Performance and readability are important.

DELIVERABLE:

Generate complete working code for:

frontend/index.html

frontend/style.css

frontend/script.js

Also explain exactly:

1. where each file should be created
2. what code goes into each file
3. how to run the frontend with the existing Flask backend
4. how to verify that the real API feed is being displayed

Do not modify backend logic.

AI-Project-Complete-Code.docx

```
===== main.py =====

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from intelligence.discovery import discover_topics
from intelligence.content_generator import generate_article

app = FastAPI(title="AI Content Creator API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def home():
    return {
        "message": "AI Content Creator API is running"
    }

@app.get("/articles")
def generate_articles(limit: int = 1):

    topics = discover_topics(limit=50)

    articles = []

    for topic in topics[:limit]:
        article = generate_article(topic)
        articles.append(article)

    return {
        "count": len(articles),
        "articles": articles
    }

===== topic_memory.json =====

[
{
    "title": "Patchloom - structured file edits for AI agents (CLI and MCP)",
    "source": "Hacker News"
},
{
    "title": "Learning When to Trust via Selective Context Preference Optimization",
    "source": "arXiv"
},
{
    "title": "$\omega$-$\theta$: A Latent Predictive World Action Model for Concurrent Humanoid Loco-Manipulation",
    "source": "arXiv"
}
```

```

},
{
  "title": "DyPES-VLA: Learning Shared Dynamics Priors and Embodiment-Specific Control for Cross-Embodiment Manipulation",
  "source": "arXiv"
},
{
  "title": "Tracing the Heart: An Evidence-Linked Pipeline for Heart-Failure Feature Engineering",
  "source": "arXiv"
},
{
  "title": "Investigating Artificial Intelligence Digital Sovereignty in Mobile Shopping Apps: A Case Study of Nigeria",
  "source": "arXiv"
},
{
  "title": "A new alloy turned up in a grain of sand from Hiroshima Bay",
  "source": "Hacker News"
},
{
  "title": "The Excellent HP OmniBook 5 Is $550 Off Its Retail Price Today",
  "source": "Hacker News"
},
{
  "title": "Agents feel free to take desktop screenshots?",
  "source": "Hacker News"
},
{
  "title": "Open-source federated AI framework for Space Solar power routing",
  "source": "Hacker News"
},
{
  "title": "Trump drops BBC's commercial arm from Panorama lawsuit, but main case remains",
  "source": "Hacker News"
},
{
  "title": "Elixir's GenStage Demand, a visual explainer",
  "source": "Hacker News"
},
{
  "title": "Apple says Mac users in China can connect to Alibaba's Qwen AI service",
  "source": "Hacker News"
}
]

```

===== content_generator.py =====

```

from typing import Dict
import json
import re
import ollama

```

```

def generate_article(topic) -> Dict:

```

```

title = topic.title

prompt = f"""
Create an informative article about this topic:

Topic: {title}

Return ONLY valid JSON.
Do not use markdown.
Do not add ```json or any explanation.

Use exactly this format:

{{
"summary": "Write a short 2-3 sentence summary.",
"key_points": [
"Point 1",
"Point 2",
"Point 3",
"Point 4"
]
}}

Keep the content factual, clear and easy to understand.
"""

response = ollama.chat(
model="llama3.2",
messages=[
{
"role": "user",
"content": prompt
}
]
)

content = response["message"]["content"].strip()

try:
generated = json.loads(content)

except json.JSONDecodeError:

# Remove markdown code fences if Ollama adds them
cleaned = re.sub(
r"```(?:json)?\s*|\s*```",
"",
content,
flags=re.IGNORECASE
).strip()

try:
generated = json.loads(cleaned)

except json.JSONDecodeError:
generated = {
"summary": content,
"key_points": []
}

```

```

}

article = {
    "title": title,
    "summary": generated.get("summary", ""),
    "key_points": generated.get("key_points", []),
    "source": topic.source,
    "source_url": topic.url,
    "category": topic.category
}

return article

===== discovery.py =====

import requests
import xml.etree.ElementTree as ET

from .models import Topic

def is_ai_topic(title):

    keywords = [
        "ai",
        "artificial intelligence",
        "machine learning",
        "llm",
        "gpt",
        "gemini",
        "claude",
        "openai",
        "anthropic",
        "google deepmind",
        "hugging face",
        "robotics",
        "neural network",
        "generative ai",
        "agent",
        "ai agent",
        "transformer",
        "deep learning"
    ]

    title_lower = title.lower()

    for keyword in keywords:

        if keyword in title_lower:
            return True

    return False

def normalize_title(title):

    stop_words = {
        "the",
        "a",
        "an",

```

```

"and",
"or",
"of",
"to",
"in",
"on",
"for",
"with",
"from",
"how",
"what",
"when",
"via",
"using",
"new",
"ai",
"model",
"models",
"learning",
"system",
"systems"
}

title = title.lower()

for character in ",.?!?-[ ]{}":
    title = title.replace(character, " ")

words = title.split()

meaningful_words = {
    word
    for word in words
    if word not in stop_words and len(word) > 2
}

return meaningful_words

def discover_topics(limit=10):
    url = "https://hacker-news.firebaseio.com/v0/newstories.json"

    response = requests.get(
        url,
        timeout=15
    )

    response.raise_for_status()

    story_ids = response.json()

    topics = []

    for story_id in story_ids[:limit]:
        story_url = (
            f"https://hacker-news.firebaseio.com/v0/item/{story_id}.json"
        )

```

```

try:

story_response = requests.get(
story_url,
timeout=10
)

story_response.raise_for_status()

story = story_response.json()

except requests.RequestException:

continue

if not story:
continue

title = story.get("title", "")

if not title:
continue

if not is_ai_topic(title):
continue

article_url = story.get(
"url",
f"https://news.ycombinator.com/item?id={story_id}"
)

topic = Topic(
title=title,
description=title,
category="Technology",
source="Hacker News",
url=article_url
)

topics.append(topic)

return topics

def discover_arxiv_topics(limit=5):

url = "https://export.arxiv.org/api/query"

params = {
"search_query": "cat:cs.AI OR cat:cs.LG OR cat:cs.RO",
"start": 0,
"max_results": limit,
"sortBy": "submittedDate",
"sortOrder": "descending"
}

try:

response = requests.get(
url,
params=params,
timeout=45

```

```

)

response.raise_for_status()

except requests.RequestException as error:

print("arXiv request failed:", error)

return []

try:

root = ET.fromstring(response.text)

except ET.ParseError:

print("Could not parse arXiv response.")

return []

namespace = {
"atom": "http://www.w3.org/2005/Atom"
}

topics = []

for entry in root.findall(
"atom:entry",
namespace
):

title_element = entry.find(
"atom:title",
namespace
)

summary_element = entry.find(
"atom:summary",
namespace
)

published_element = entry.find(
"atom:published",
namespace
)

link_element = entry.find(
"atom:id",
namespace
)

if title_element is None:
continue

if not title_element.text:
continue

title = " ".join(
title_element.text.strip().split()
)

description = ""

```



```

if (
summary_element is not None
and summary_element.text
):

description = " ".join(
summary_element.text.strip().split()
)

published_at = None

if (
published_element is not None
and published_element.text
):

published_at = published_element.text

article_url = ""

if (
link_element is not None
and link_element.text
):

article_url = link_element.text.strip()

topic = Topic(
title=title,
description=description,
category="AI Research",
source="arXiv",
url=article_url,
published_at=published_at
)

topics.append(topic)

return topics

===== editorial_brain.py =====

def make_editorial_decision(
trend_score,
source_confidence,
technical_relevance,
is_duplicate
):

# Check if this topic was already covered
if is_duplicate:
return {
"decision": "REJECT",
"reason": "Similar topic was already covered recently."
}

# Check trend score
if trend_score < 50:
return {

```

```

"decision": "REJECT",
"reason": "Trend score is too low."
}

# Check source confidence
if source_confidence < 50:
    return {
        "decision": "REJECT",
        "reason": "Source confidence is too low."
    }

# Check technical relevance
if technical_relevance < 40:
    return {
        "decision": "REJECT",
        "reason": "Technical relevance is too low."
    }

# All checks passed
return {
    "decision": "ACCEPT",
    "reason": "Topic has strong relevance and sufficient evidence."
}

===== intelligence_pipeline.py =====

from .discovery import (
    discover_topics,
    discover_arxiv_topics,
    normalize_title
)

from .trend_analyzer import calculate_trend_score
from .source_validator import calculate_source_confidence
from .editorial_brain import make_editorial_decision
from .memory import is_duplicate_topic, remember_topic
from .content_generator import generate_article

def has_cross_source_match(topic, other_topics):

    topic_words = normalize_title(topic.title)

    for other_topic in other_topics:

        other_words = normalize_title(other_topic.title)

        common_words = topic_words.intersection(other_words)

        if len(common_words) >= 2:
            return True

    return False

def run_intelligence_pipeline():

    hacker_news_topics = discover_topics(limit=20)

    arxiv_topics = discover_arxiv_topics(limit=5)

    topics = hacker_news_topics + arxiv_topics

```

```

results = []

for topic in topics:

    if topic.source == "Hacker News":

        other_topics = arxiv_topics

        source_strength = 80
        technical_importance = 75
        community_interest = 85

    else:

        other_topics = hacker_news_topics

        source_strength = 95
        technical_importance = 95
        community_interest = 70

    cross_source_match = has_cross_source_match(
        topic,
        other_topics
    )

    duplicate = is_duplicate_topic(
        topic.title
    )

    recency = 90

    if cross_source_match:

        cross_source_confirmation = 100
        source_count = 2

    else:

        cross_source_confirmation = 40
        source_count = 1

    trend_score = calculate_trend_score(
        recency=recency,
        source_strength=source_strength,
        community_interest=community_interest,
        technical_importance=technical_importance,
        cross_source_confirmation=cross_source_confirmation
    )

    source_confidence = calculate_source_confidence(
        source_count,
        source_strength
    )

    editorial_result = make_editorial_decision(
        trend_score=trend_score,
        source_confidence=source_confidence,
        technical_relevance=technical_importance,
        is_duplicate=duplicate
    )

```

```

article = None

if editorial_result["decision"] == "ACCEPT":

    article = generate_article(topic)

    remember_topic(
        title=topic.title,
        source=topic.source
    )

    result = {
        "topic": topic.title,
        "category": topic.category,
        "source": topic.source,
        "trend_score": trend_score,
        "source_confidence": source_confidence,
        "decision": editorial_result["decision"],
        "reason": editorial_result["reason"],
        "article": article
    }

    results.append(result)

return results

===== memory.py =====

import json
import os

from .discovery import normalize_title

MEMORY_FILE = os.path.join(
    os.path.dirname(os.path.dirname(__file__)),
    "data",
    "topic_memory.json"
)

def load_memory():

    if not os.path.exists(MEMORY_FILE):
        return []

    with open(
        MEMORY_FILE,
        "r",
        encoding="utf-8"
    ) as file:

        return json.load(file)

def save_memory(memory):

    os.makedirs(
        os.path.dirname(MEMORY_FILE),
        exist_ok=True
    )

    with open(

```

```

MEMORY_FILE,
"w",
encoding="utf-8"
) as file:

json.dump(
memory,
file,
indent=4,
ensure_ascii=False
)

def is_duplicate_topic(title):

memory = load_memory()

current_words = normalize_title(title)

for old_topic in memory:

old_words = normalize_title(
old_topic["title"]
)

common_words = current_words.intersection(
old_words
)

if len(common_words) >= 3:
return True

return False

def remember_topic(title, source):

memory = load_memory()

memory.append({
"title": title,
"source": source
})

save_memory(memory)

===== models.py =====

from pydantic import BaseModel
from typing import Optional

class Topic(BaseModel):
title: str
description: str
category: str
source: str
url: str
published_at: Optional[str] = None

class IntelligenceResult(BaseModel):
topic: Topic
trend_score: float

```

```
community_interest: float
source_confidence: float
decision: str
reason: str
```

```
===== source_validator.py =====
```

```
def calculate_source_confidence(
    source_count,
    source_strength=0
):
```

```
    if source_count >= 5:
        return 95
```

```
    elif source_count == 4:
        return 85
```

```
    elif source_count == 3:
        return 75
```

```
    elif source_count == 2:
        return 60
```

```
    elif source_count == 1:
```

```
        if source_strength >= 90:
            return 70
```

```
        elif source_strength >= 80:
            return 60
```

```
        elif source_strength >= 70:
            return 50
```

```
    else:
        return 30
```

```
    return 0
```

```
===== trend_analyzer.py =====
```

```
def calculate_trend_score(
    recency,
    source_strength,
    community_interest,
    technical_importance,
    cross_source_confirmation
):
```

```
    score = (
        recency * 0.20
        + source_strength * 0.20
        + community_interest * 0.25
        + technical_importance * 0.20
        + cross_source_confirmation * 0.15
    )
```

```
    return round(score, 2)
```

```
def get_trend_level(score):
```

```

if score >= 90:
    return "VERY HIGH"

elif score >= 75:
    return "HIGH"

elif score >= 50:
    return "MEDIUM"

else:
    return "LOW"

===== __init__.py =====

===== test_article.py =====

from intelligence.discovery import discover_arxiv_topics
from intelligence.content_generator import generate_article

print("Finding fresh arXiv topic...")

topics = discover_arxiv_topics(limit=1)

if not topics:

    print("No topic found.")

else:

    topic = topics[0]

    print("\nTopic found:")
    print(topic.title)

    article = generate_article(topic)

    print("\n===== GENERATED ARTICLE =====")

    print("\nTITLE:")
    print(article["title"])

    print("\nSUMMARY:")
    print(article["summary"])

    print("\nKEY POINTS:")

    for point in article["key_points"]:
        print("-", point)

    print("\nSOURCE:")
    print(article["source"])

    print("\nSOURCE URL:")
    print(article["source_url"])

    print("\nCATEGORY:")
    print(article["category"])

===== test_content_generator.py =====

from intelligence.content_generator import generate_article
from intelligence.discovery import discover_topics

print("Starting content generator test...")

```

```

topics = discover_topics(limit=50)

if not topics:

    print("No topics found.")

else:

    topic = topics[0]

    article = generate_article(topic)

    print("\n=====")
    print("TITLE:")
    print(article["title"])

    print("\nSUMMARY:")
    print(article["summary"])

    print("\nKEY POINTS:")

    for point in article["key_points"]:
        print("-", point)

    print("\nSOURCE:")
    print(article["source"])

    print("\nURL:")
    print(article["source_url"])

    print("\nCATEGORY:")
    print(article["category"])

===== test_discovery.py =====

from intelligence.discovery import discover_arxiv_topics

print("Starting arXiv test...")

topics = discover_arxiv_topics(limit=5)

print("Topics discovered:", len(topics))

for topic in topics:
    print("Title:", topic.title)

===== test_editorial.py =====

from intelligence.editorial_brain import make_editorial_decision

result = make_editorial_decision(
    trend_score=90,
    source_confidence=95,
    technical_relevance=90,
    is_duplicate=True
)

print("Decision:", result["decision"])
print("Reason:", result["reason"])

===== test_pipeline.py =====

from intelligence.intelligence_pipeline import run_intelligence_pipeline

```



```

results = run_intelligence_pipeline()

for result in results:

print("\n-----")

print("Topic:", result["topic"])
print("Category:", result["category"])
print("Source:", result["source"])
print("Trend Score:", result["trend_score"])
print("Source Confidence:", result["source_confidence"])
print("Decision:", result["decision"])
print("Reason:", result["reason"])

article = result.get("article")

if article:

print("\n===== GENERATED ARTICLE =====")

print("\nTITLE:")
print(article["title"])

print("\nSUMMARY:")
print(article["summary"])

print("\nKEY POINTS:")

for point in article["key_points"]:
print("-", point)

print("\nSOURCE:")
print(article["source"])

print("\nSOURCE URL:")
print(article["source_url"])

print("\nCATEGORY:")
print(article["category"])

===== test_source.py =====

from intelligence.source_validator import calculate_source_confidence

confidence = calculate_source_confidence(5)

print("Source Confidence:", confidence)

===== test_trend.py =====

from intelligence.trend_analyzer import (
calculate_trend_score,
get_trend_level
)

score = calculate_trend_score(
recency=95,
source_strength=90,
community_interest=88,
technical_importance=92,
cross_source_confirmation=85
)

```

```

level = get_trend_level(score)

print("Trend Score:", score)
print("Trend Level:", level)

===== first.html =====

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>NeuraPress – AI Content Intelligence</title>

<style>
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

:root {
--bg: #050711;
--card: rgba(15, 20, 38, 0.72);
--border: rgba(148, 163, 184, 0.14);
--text: #f8fafc;
--muted: #8d99ae;
--purple: #8b5cf6;
--cyan: #22d3ee;
--green: #34d399;
}

html {
scroll-behavior: smooth;
}

body {
min-height: 100vh;
color: var(--text);
font-family: Inter, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
background:
radial-gradient(circle at 10% 10%, rgba(139,92,246,.20), transparent 28%),
radial-gradient(circle at 90% 10%, rgba(34,211,238,.14), transparent 25%),
radial-gradient(circle at 50% 80%, rgba(99,102,241,.08), transparent 30%),
var(--bg);
overflow-x: hidden;
}

/* Animated background */

body::before {
content: "";
position: fixed;
inset: 0;
pointer-events: none;

```

```

background-image:
  linear-gradient(rgba(255,255,255,.018) 1px, transparent 1px),
  linear-gradient(90deg, rgba(255,255,255,.018) 1px, transparent 1px);
background-size: 55px 55px;
mask-image: linear-gradient(to bottom, black, transparent 85%);
}

.orb {
position: fixed;
width: 280px;
height: 280px;
border-radius: 50%;
filter: blur(100px);
pointer-events: none;
z-index: -1;
}

.orb-one {
background: rgba(124,58,237,.15);
left: -120px;
top: 25%;
animation: float 9s ease-in-out infinite;
}

.orb-two {
background: rgba(6,182,212,.12);
right: -100px;
top: 55%;
animation: float 11s ease-in-out infinite reverse;
}

@keyframes float {
50% {
transform: translateY(-35px) translateX(25px);
}
}

/* NAVBAR */

nav {
height: 76px;
padding: 0 6%;
display: flex;
align-items: center;
justify-content: space-between;
position: sticky;
top: 0;
z-index: 100;
border-bottom: 1px solid var(--border);
background: rgba(5,7,17,.72);
backdrop-filter: blur(22px);
}

.brand {
display: flex;

```

```
align-items: center;
gap: 12px;
}

.logo {
width: 43px;
height: 43px;
display: grid;
place-items: center;
border-radius: 13px;
font-size: 21px;
background: linear-gradient(135deg, #7c3aed, #06b6d4);
box-shadow: 0 0 35px rgba(124,58,237,.35);
}

.brand-name {
font-size: 19px;
font-weight: 850;
letter-spacing: -.5px;
}

.brand-subtitle {
color: #64748b;
font-size: 9px;
letter-spacing: 1.5px;
text-transform: uppercase;
}

.status {
display: flex;
align-items: center;
gap: 8px;
padding: 9px 14px;
border-radius: 999px;
border: 1px solid rgba(52,211,153,.18);
background: rgba(52,211,153,.06);
color: #86efac;
font-size: 12px;
font-weight: 700;
}

.status-dot {
width: 8px;
height: 8px;
border-radius: 50%;
background: var(--green);
box-shadow: 0 0 14px var(--green);
animation: pulse 1.8s infinite;
}

@keyframes pulse {
50% {
opacity: .35;
transform: scale(.75);
```

```
}  
}  
  
/* MAIN */  
  
main {  
width: min(1180px, 92%);  
margin: auto;  
}  
  
/* HERO */  
  
.hero {  
min-height: 530px;  
padding: 85px 0 45px;  
display: grid;  
grid-template-columns: 1.25fr .75fr;  
align-items: center;  
gap: 40px;  
}  
  
.eyebrow {  
width: fit-content;  
padding: 8px 14px;  
margin-bottom: 22px;  
border: 1px solid rgba(167,139,250,.18);  
border-radius: 999px;  
background: rgba(139,92,246,.07);  
color: #c4b5fd;  
font-size: 11px;  
font-weight: 800;  
letter-spacing: 1px;  
text-transform: uppercase;  
}  
  
.hero h1 {  
font-size: clamp(48px, 7vw, 82px);  
line-height: .96;  
letter-spacing: -5px;  
}  
  
.gradient {  
background: linear-gradient(  
100deg,  
#c4b5fd,  
#67e8f9,  
#a78bfa  
);  
background-size: 220% auto;  
-webkit-background-clip: text;  
background-clip: text;  
color: transparent;  
animation: gradientMove 6s linear infinite;  
}
```

```

@keyframes gradientMove {
  to {
    background-position: 220% center;
  }
}

.hero p {
  max-width: 650px;
  margin-top: 25px;
  color: var(--muted);
  font-size: 17px;
  line-height: 1.8;
}

/* AI CORE */

.hero-visual {
  min-height: 300px;
  display: grid;
  place-items: center;
}

.ai-core {
  width: 175px;
  height: 175px;
  display: grid;
  place-items: center;
  position: relative;
  border-radius: 50%;
  border: 1px solid rgba(139,92,246,.3);
  background:
    radial-gradient(circle, rgba(34,211,238,.15), transparent 42%),
    rgba(139,92,246,.05);
  box-shadow:
    0 0 0 20px rgba(139,92,246,.025),
    0 0 90px rgba(139,92,246,.22),
    inset 0 0 50px rgba(34,211,238,.08);
  animation: corePulse 5s ease-in-out infinite;
}

.ai-core::before {
  content: "";
  position: absolute;
  width: 230px;
  height: 230px;
  border-radius: 50%;
  border: 1px solid rgba(34,211,238,.16);
  animation: spin 14s linear infinite;
}

.ai-core::after {
  content: "";
  position: absolute;
  width: 280px;

```

```
height: 280px;
border-radius: 50%;
border: 1px dashed rgba(139,92,246,.16);
animation: spin 22s linear infinite reverse;
}

.core-icon {
font-size: 55px;
filter: drop-shadow(0 0 20px rgba(103,232,249,.55));
}

@keyframes corePulse {
50% {
transform: scale(1.04);
}
}

@keyframes spin {
to {
transform: rotate(360deg);
}
}

/* STATS */

.stats {
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 14px;
margin: 10px 0 30px;
}

.stat {
padding: 21px;
border: 1px solid var(--border);
border-radius: 18px;
background: rgba(255,255,255,.028);
backdrop-filter: blur(18px);
transition: .3s ease;
}

.stat:hover {
transform: translateY(-4px);
border-color: rgba(139,92,246,.30);
background: rgba(255,255,255,.045);
}

.stat-top {
display: flex;
justify-content: space-between;
color: #68758a;
font-size: 10px;
letter-spacing: 1.2px;
text-transform: uppercase;
}
```

```
.stat-icon {
color: #a78bfa;
}

.stat-value {
margin-top: 12px;
font-size: 26px;
font-weight: 850;
}

.stat-note {
margin-top: 4px;
color: #64748b;
font-size: 11px;
}

/* WORKSPACE */

.workspace {
display: grid;
grid-template-columns: 1fr 300px;
gap: 18px;
margin-bottom: 25px;
}

.generator,
.pipeline-card {
border: 1px solid var(--border);
border-radius: 24px;
background:
linear-gradient(
145deg,
rgba(255,255,255,.055),
rgba(255,255,255,.018)
);
backdrop-filter: blur(22px);
box-shadow: 0 25px 80px rgba(0,0,0,.25);
}

.generator {
padding: 30px;
}

.section-label {
color: #8b98ac;
font-size: 10px;
font-weight: 800;
letter-spacing: 1.5px;
text-transform: uppercase;
}

.generator h2 {
margin-top: 8px;
font-size: 25px;
letter-spacing: -.8px;
```



```
}

.description {
margin-top: 8px;
color: #718096;
font-size: 13px;
line-height: 1.6;
}

.generate-row {
margin-top: 23px;
padding: 10px;
display: flex;
align-items: center;
justify-content: space-between;
gap: 15px;
border: 1px solid var(--border);
border-radius: 17px;
background: rgba(0,0,0,.16);
}

.engine {
display: flex;
align-items: center;
gap: 12px;
padding-left: 6px;
}

.engine-icon {
width: 39px;
height: 39px;
display: grid;
place-items: center;
border-radius: 11px;
background: rgba(139,92,246,.12);
border: 1px solid rgba(139,92,246,.18);
}

.engine strong {
display: block;
font-size: 13px;
}

.engine span {
display: block;
margin-top: 3px;
color: #64748b;
font-size: 10px;
}

.generate-btn {
padding: 14px 22px;
border: none;
border-radius: 12px;
color: white;
```

```
background: linear-gradient(135deg, #7c3aed, #0891b2);
box-shadow:
  0 12px 30px rgba(124,58,237,.24),
  inset 0 1px rgba(255,255,255,.25);
font-weight: 800;
cursor: pointer;
transition: .25s ease;
}

.generate-btn:hover {
transform: translateY(-2px);
box-shadow: 0 18px 40px rgba(124,58,237,.34);
}

.generate-btn:active {
transform: scale(.98);
}

.generate-btn:disabled {
opacity: .55;
cursor: not-allowed;
transform: none;
}

/* PIPELINE */

.pipeline-card {
padding: 25px;
}

.pipeline-card h3 {
margin-top: 7px;
font-size: 18px;
}

.pipeline {
margin-top: 20px;
display: grid;
gap: 15px;
}

.pipeline-item {
display: flex;
align-items: flex-start;
gap: 11px;
}

.pipeline-number {
width: 27px;
height: 27px;
flex: 0 0 27px;
display: grid;
place-items: center;
border-radius: 9px;
color: #c4b5fd;
background: rgba(139,92,246,.09);
```

```
border: 1px solid rgba(139,92,246,.16);
font-size: 9px;
font-weight: 800;
}

.pipeline-item strong {
display: block;
font-size: 12px;
}

.pipeline-item span {
display: block;
margin-top: 3px;
color: #64748b;
font-size: 10px;
}

/* STATUS */

#status {
min-height: 28px;
margin: 18px 0;
display: flex;
justify-content: center;
}

.loading {
display: inline-flex;
align-items: center;
gap: 10px;
padding: 10px 16px;
border: 1px solid rgba(139,92,246,.18);
border-radius: 999px;
background: rgba(139,92,246,.07);
color: #c4b5fd;
font-size: 12px;
font-weight: 700;
}

.loader {
width: 15px;
height: 15px;
border: 2px solid rgba(255,255,255,.18);
border-top-color: #a78bfa;
border-radius: 50%;
animation: spin .75s linear infinite;
}

/* RESULTS */

#articles {
padding-bottom: 85px;
}

.results-title {
margin: 35px 0 15px;
```

```
}

.results-title h2 {
margin-top: 5px;
font-size: 24px;
}

.results-title p {
margin-top: 5px;
color: #64748b;
font-size: 11px;
}

.article {
position: relative;
overflow: hidden;
padding: 34px;
border: 1px solid var(--border);
border-radius: 25px;
background:
linear-gradient(
145deg,
rgba(255,255,255,.06),
rgba(255,255,255,.018)
);
box-shadow: 0 25px 80px rgba(0,0,0,.30);
animation: articleIn .65s ease both;
}

.article::before {
content: "";
position: absolute;
width: 260px;
height: 260px;
right: -130px;
top: -140px;
border-radius: 50%;
background: rgba(34,211,238,.08);
filter: blur(35px);
}

@keyframes articleIn {
from {
opacity: 0;
transform: translateY(25px);
}

to {
opacity: 1;
transform: translateY(0);
}
}

.article-label {
display: inline-flex;
```

```
padding: 7px 11px;
border-radius: 999px;
color: #a78bfa;
background: rgba(139,92,246,.08);
border: 1px solid rgba(139,92,246,.14);
font-size: 10px;
font-weight: 800;
letter-spacing: 1px;
text-transform: uppercase;
}

.article h2 {
margin-top: 18px;
font-size: clamp(28px, 4vw, 44px);
line-height: 1.08;
letter-spacing: -1.7px;
}

.section-title {
margin: 28px 0 10px;
color: #c4b5fd;
font-size: 11px;
font-weight: 800;
letter-spacing: 1px;
text-transform: uppercase;
}

.summary {
color: #b5c0d2;
font-size: 15px;
line-height: 1.85;
}

.article ul {
list-style: none;
display: grid;
gap: 9px;
}

.article li {
position: relative;
padding: 13px 15px 13px 35px;
border: 1px solid rgba(148,163,184,.08);
border-radius: 13px;
background: rgba(255,255,255,.025);
color: #b8c2d2;
font-size: 14px;
line-height: 1.55;
}

.article li::before {
content: "◆";
position: absolute;
left: 14px;
```

```
color: var(--cyan);
}

.metadata {
display: flex;
flex-wrap: wrap;
align-items: center;
gap: 8px;
margin-top: 28px;
padding-top: 20px;
border-top: 1px solid var(--border);
}

.tag {
padding: 7px 11px;
border-radius: 999px;
color: #aeb9cb;
background: rgba(255,255,255,.035);
border: 1px solid rgba(148,163,184,.1);
font-size: 11px;
}

.source-link {
margin-left: auto;
padding: 9px 14px;
border-radius: 10px;
color: #67e8f9;
background: rgba(34,211,238,.07);
border: 1px solid rgba(34,211,238,.12);
text-decoration: none;
font-size: 11px;
font-weight: 800;
transition: .2s;
}

.source-link:hover {
transform: translateY(-1px);
background: rgba(34,211,238,.13);
}

.empty {
padding: 35px;
text-align: center;
color: #8290a5;
border: 1px dashed rgba(148,163,184,.16);
border-radius: 18px;
background: rgba(255,255,255,.018);
line-height: 1.7;
}

footer {
padding: 25px 6%;
border-top: 1px solid var(--border);
color: #4f5b70;
```

```
text-align: center;
font-size: 10px;
letter-spacing: .7px;
}

/* RESPONSIVE */

@media (max-width: 850px) {

  .hero {
    grid-template-columns: 1fr;
    padding-top: 60px;
  }

  .hero-visual {
    display: none;
  }

  .workspace {
    grid-template-columns: 1fr;
  }
}

@media (max-width: 620px) {

  nav {
    padding: 0 20px;
  }

  .status {
    display: none;
  }

  main {
    width: 94%;
  }

  .hero h1 {
    font-size: 50px;
    letter-spacing: -3px;
  }

  .hero p {
    font-size: 15px;
  }

  .stats {
    grid-template-columns: 1fr;
  }

  .generator {
    padding: 22px;
  }

  .generate-row {
    flex-direction: column;
    align-items: stretch;
  }

  .generate-btn {
```

```
width: 100%;
}

.article {
padding: 24px;
}

.source-link {
width: 100%;
margin-left: 0;
text-align: center;
}
}
</style>
</head>

<body>

<div class="orb orb-one"></div>
<div class="orb orb-two"></div>

<!-- NAVBAR -->

<nav>

<div class="brand">

<div class="logo">◆</div>

<div>
<div class="brand-name">NeuraPress</div>
<div class="brand-subtitle">
AI Content Intelligence
</div>
</div>

</div>

<div class="status">
<span class="status-dot"></span>
AI Engine Online
</div>

</nav>

<main>

<!-- HERO -->

<section class="hero">

<div>

<div class="eyebrow">
■ Autonomous Intelligence Platform
</div>

<h1>
From
<span class="gradient">Signals</span>
<br>
```


to Stories.

</h1>

<p>

Discover emerging AI trends, understand what matters,
and transform real-world information into intelligent,
publication-ready stories using AI.

</p>

</div>

<div class="hero-visual">

<div class="ai-core">

<div class="core-icon">

◆

</div>

</div>

</div>

</section>

<!-- STATS -->

<section class="stats">

<div class="stat">

<div class="stat-top">

Intelligence

■

</div>

<div class="stat-value">

LIVE

</div>

<div class="stat-note">

Real-time topic discovery

</div>

</div>

<div class="stat">

<div class="stat-top">

AI Engine

◆

</div>

<div class="stat-value">

LOCAL

</div>

<div class="stat-note">

Powered by local LLM

</div>

</div>

```
<div class="stat">

<div class="stat-top">
<span>Output</span>
<span class="stat-icon">■</span>
</div>

<div class="stat-value">
READY
</div>

<div class="stat-note">
Publication-ready content
</div>

</div>

</section>

<!-- GENERATOR -->

<section class="workspace">

<div class="generator">

<div class="section-label">
Intelligence Workspace
</div>

<h2>
Generate your next story
</h2>

<p class="description">
Let NeuraPress research the latest signals and turn
them into structured AI-generated content.
</p>

<div class="generate-row">

<div class="engine">

<div class="engine-icon">
■
</div>

<div>

<strong>
Autonomous Content Generator
</strong>

<span>
Research → Analyze → Write
</span>

</div>

</div>

<button
```

```
id="generateButton"
class="generate-btn"
onclick="generateArticles()">
```

■ Generate Article

```
</button>
```

```
</div>
```

```
</div>
```

```
<!-- PIPELINE -->
```

```
<aside class="pipeline-card">
```

```
<div class="section-label">
```

Pipeline

```
</div>
```

```
<h3>
```

How it works

```
</h3>
```

```
<div class="pipeline">
```

```
<div class="pipeline-item">
```

```
<div class="pipeline-number">
```

01

```
</div>
```

```
<div>
```

```
<strong>
```

Discover

```
</strong>
```

```
<span>
```

Find emerging signals

```
</span>
```

```
</div>
```

```
</div>
```

```
<div class="pipeline-item">
```

```
<div class="pipeline-number">
```

02

```
</div>
```

```
<div>
```

```
<strong>
```

Understand

```
</strong>
```

```
<span>
```

Analyze relevant context

```
</span>
```

```
</div>
```

```
</div>

<div class="pipeline-item">

<div class="pipeline-number">
03
</div>

<div>

<strong>
Generate
</strong>

<span>
Craft intelligent stories
</span>

</div>

</div>

</div>

</div>

</aside>

</section>

<div id="status"></div>

<!-- OUTPUT -->

<section id="articles">

<div class="results-title">

<div class="section-label">
AI Output
</div>

<h2>
Latest Intelligence
</h2>

<p>
AI-generated insights will appear here.
</p>

</div>

</section>

</main>

<footer>
NEURAPRESS • AUTONOMOUS AI CONTENT INTELLIGENCE
</footer>

<script>

async function generateArticles() {

const status =
document.getElementById("status");
```

```

const articlesDiv =
document.getElementById("articles");

const button =
document.getElementById("generateButton");

status.innerHTML = `
<div class="loading">

<span class="loader"></span>

AI is researching & writing...

</div>
`;

const oldArticle =
articlesDiv.querySelector(".article");

if (oldArticle) {
oldArticle.remove();
}

button.disabled = true;
button.innerText = "Generating...";

try {

const response = await fetch(
"http://127.0.0.1:8000/articles?limit=1"
);

if (!response.ok) {
throw new Error("API request failed");
}

const data =
await response.json();

if (data.count === 0) {

status.innerHTML = `
<div class="empty">

No topics available right now.

</div>
`;

return;
}

status.innerHTML = "";

data.articles.forEach(article => {

const div =
document.createElement("div");

div.className = "article";

div.innerHTML = `

```

```
<div class="article-label">
♦ AI Generated Intelligence
</div>

<h2>
${article.title}
</h2>

<div class="section-title">
■ Executive Summary
</div>

<p class="summary">
${article.summary}
</p>

<div class="section-title">
♦ Key Insights
</div>

<ul>

${article.key_points
.map(point =>
`<li>${point}</li>`
)
.join("")}

</ul>

<div class="metadata">

<span class="tag">
■ ${article.source}
</span>

<span class="tag">
■■ ${article.category}
</span>

<a
class="source-link"
href="${article.source_url}"
target="_blank"
rel="noopener noreferrer">

Read Original →

</a>

</div>

`;

articlesDiv.appendChild(div);

setTimeout(() => {

div.scrollIntoView({
```

```

behavior: "smooth",
block: "start"
});

}, 100);

});

} catch (error) {

console.error(error);

status.innerHTML = `
<div class="empty">

■■ Unable to connect to AI backend.

<br>

<small>
Make sure FastAPI is running on port 8000.
</small>

</div>
`;

} finally {

button.disabled = false;

button.innerText =
"■ Generate Article";

}

}

</script>

</body>

</html>

===== make_word.py =====

from pathlib import Path
from docx import Document
from docx.shared import Pt

# Project folder
project = Path(__file__).parent

# Create Word document
doc = Document()

# Title
doc.add_heading("AI Project - Complete Source Code", 0)
doc.add_paragraph(
"Complete source code automatically collected from the project."
)

# Files to include
extensions = {
".py",

```

```

".html",
".css",
".js",
".json",
".txt"
}

# Folders to ignore
ignore_folders = {
".git",
".venv",
"venv",
"__pycache__",
"node_modules"
}

# Files to ignore
ignore_files = {
"make_word.py",
"AI-Project-Complete-Code.docx"
}

# Find files
files = []

for file in project.rglob("**"):
    if not file.is_file():
        continue

    if file.name in ignore_files:
        continue

    if any(folder in file.parts for folder in ignore_folders):
        continue

    if file.suffix.lower() in extensions:
        files.append(file)

files.sort()

# Project structure
doc.add_heading("Project Structure", level=1)

for file in files:
    relative_path = file.relative_to(project)

    p = doc.add_paragraph()

    run = p.add_run(str(relative_path))

    run.font.name = "Consolas"
    run.font.size = Pt(9)

# Page break
doc.add_page_break()

# Source code
doc.add_heading("Complete Source Code", level=1)

```



```

for number, file in enumerate(files, start=1):

    relative_path = file.relative_to(project)

    doc.add_heading(
        f"{number}. {relative_path}",
        level=2
    )

    try:
        code = file.read_text(
            encoding="utf-8",
            errors="replace"
        )
    except Exception as error:

        doc.add_paragraph(
            f"Could not read file: {error}"
        )

    continue

# IMPORTANT:
# Add the complete code as ONE paragraph.
# This is much faster than adding every line separately.

paragraph = doc.add_paragraph()

run = paragraph.add_run(code)

run.font.name = "Consolas"
run.font.size = Pt(8)

# Separator
doc.add_paragraph(
    "-----"
)

# Save
output_file = project / "AI-Project-Complete-Code.docx"

doc.save(output_file)

print()
print("=" * 60)
print("WORD FILE CREATED SUCCESSFULLY!")
print("=" * 60)
print()
print("Files included:", len(files))
print("Location:")
print(output_file)
print()
print("=" * 60)

```

PROMPT—1

MASTER PROMPT — AURA Autonomous Publishing Module

You are a **Senior Python Backend Engineer, AI Systems Architect, and Production API Engineer** with expertise in FastAPI, asynchronous Python, and AI/ML.

I am participating in an **AI Hackathon**.

Our project is:

AURA — Autonomous AI Technology Persona

Tagline:

> "Not an AI Writer. An AI Technology Thinker."

There are **three team members**, and the responsibilities are strictly separated.

TEAM RESPONSIBILITIES

Member 1 — Persona & Intelligence Layer

Member 1 is responsible for:

- * Persona creation
- * Persona configuration
- * Memory
- * Opinion Engine
- * Persona database
- * Personality/context management

Member 1 exposes:

```
```http
GET /api/persona
```
```

Member 2 — Topic Discovery Layer

Member 2 is responsible for:

- * Discovering AI/technology topics from the internet
- * Researching topics
- * Ranking topics

- * Deciding which topics are worth publishing
- * Approving topics
- * Providing source URLs

Member 2 exposes:

```
```http
GET /api/topics/approved
```
```

Member 3 — MY RESPONSIBILITY

I am ****Member 3****.

My responsibility is ****ONLY** the Autonomous Publishing Module**.

I must NOT implement:

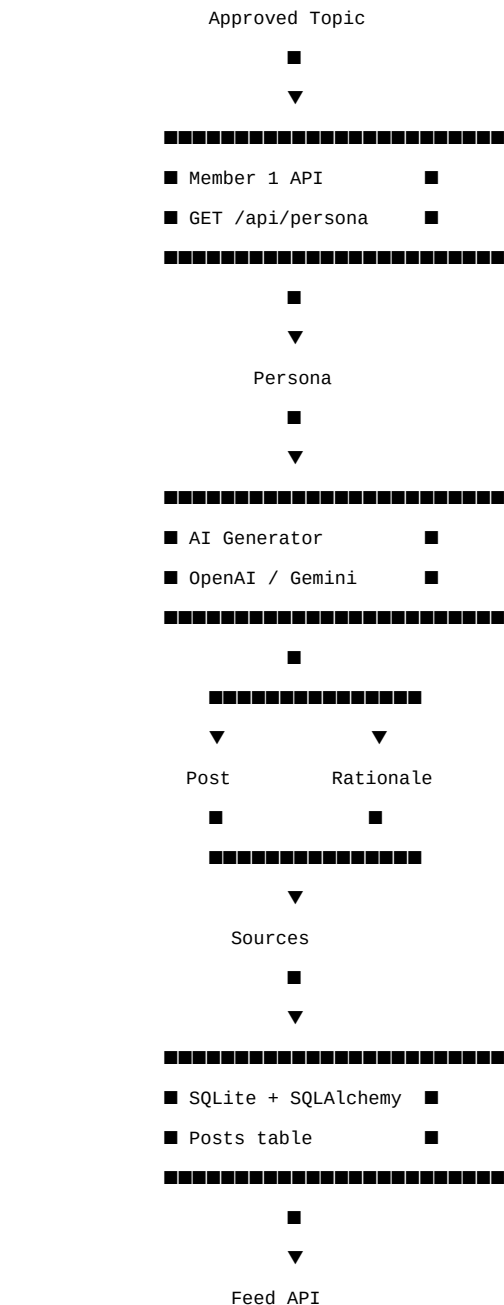
- * Topic discovery
- * Internet crawling
- * Web scraping
- * Topic ranking
- * Topic approval logic
- * Persona creation
- * Memory
- * Opinion Engine
- * Persona database

My service must simply ****consume the approved topic and persona from Member 1 and Member 2**** and autonomously generate and persist the final output.

CORE SYSTEM FLOW

The complete workflow must be:

[illegible]



...

REQUIRED TECHNOLOGY STACK

Use:

- * Python 3.12+
- * FastAPI
- * Uvicorn
- * APScheduler
- * SQLAlchemy 2.x
- * SQLite
- * aiosqlite
- * Pydantic v2
- * pydantic-settings

- * httpx
- * OpenAI SDK
- * Google Gemini SDK
- * Docker
- * Environment variables
- * Async/await throughout the application

REQUIRED FEATURES

The application MUST include:

1. FastAPI backend
2. APScheduler
3. SQLite
4. SQLAlchemy ORM
5. Async SQLAlchemy
6. OpenAI integration
7. Gemini integration
8. OpenAI/Gemini fallback strategy
9. Feed API
10. Dashboard API
11. Manual publish API
12. Health-check API
13. Structured logging
14. Error handling
15. External API timeout handling
16. External API failure handling
17. AI generation failure handling
18. Database rollback handling
19. Pydantic request/response models
20. Dependency Injection
21. Modular folder architecture
22. Docker support
23. requirements.txt
24. .env.example
25. README.md
26. Type hints
27. Professional naming
28. Comments where useful
29. Production-oriented configuration
30. Graceful startup/shutdown

AUTONOMOUS SCHEDULER

Use APScheduler.

The scheduler must execute the publishing workflow:

```
```text
Every 30 minutes
 ↓
Fetch approved topic
 ↓
Fetch persona
 ↓
Generate post
 ↓
Generate rationale
 ↓
Attach sources
 ↓
Save post
```
```

The scheduler must start automatically when FastAPI starts.

The scheduler must stop gracefully when FastAPI shuts down.

Use an interval trigger.

Default configuration:

```
```env
SCHEDULER_ENABLED=true
SCHEDULER_INTERVAL_MINUTES=30
```
```

Do not hardcode the interval.

Read it from environment variables.

Use:

- * max_instances=1
- * coalesce=True
- * reasonable misfire_grace_time
- * proper logging

Prevent overlapping publishing jobs.

MEMBER 2 API INTEGRATION

Member 2 provides approved topics.

Call:

```
```http
GET /api/topics/approved
```
```

The URL must be configurable:

```
```env
MEMBER2_TOPICS_URL=http://localhost:8001/api/topics/approved
```
```

Support a clean response such as:

```
```json
{
 "topic": {
 "title": "AI Agents in Software Development",
 "description": "How autonomous coding agents are changing software development.",
 "sources": [
 "https://example.com/article"
]
 }
}
```
```

Also make the publisher robust enough to handle reasonable variations such as:

```
```json
{
 "topics": [
 {
 "title": "AI Agents in Software Development",
 "description": "..."
 }
]
}
```
```

or:

```
```json
{
 "title": "AI Agents in Software Development",
 "description": "..."
}
```

```
}
...

```

If there is no approved topic:

- \* Do not generate a post
- \* Do not insert an empty post
- \* Log the condition
- \* End the scheduler cycle normally

```

```

## MEMBER 1 API INTEGRATION

Member 1 provides the current persona.

Call:

```
```http  
GET /api/persona  
...  

```

Configurable URL:

```
```env  
MEMBER1_PERSONA_URL=http://localhost:8002/api/persona
...

```

Recommended response:

```
```json  
{  
  "persona": {  
    "name": "AURA",  
    "role": "AI Technology Thinker",  
    "tone": "analytical",  
    "style": "concise and insightful",  
    "values": [  
      "technical accuracy",  
      "innovation",  
      "practical thinking"  
    ],  
    "opinions": [  
      "AI should augment human reasoning"  
    ]  
  }  
}  
...  

```

Do NOT create persona logic inside this service.

Treat the persona as external input.

AI GENERATOR

Create:

```
```text
app/generator.py
```
```

Required function:

```
```python
async def generate_post(
 topic: dict,
 persona: dict
) -> dict:
 ...
```
```

Expected output:

```
```json
{
 "text": "...",
 "rationale": "...",
 "sources": []
}
```
```

The AI must generate:

1. LinkedIn-style post

The post should:

- * Be technology-focused
- * Be insightful
- * Have a clear point of view
- * Follow the supplied persona
- * Avoid generic motivational content
- * Avoid excessive emojis
- * Avoid fake personal experiences
- * Avoid fabricated statistics
- * Avoid fabricated sources
- * Avoid claiming unsupported facts
- * Be suitable for a professional technology audience

2. Rationale

The rationale should explain:

- * Why the topic is worth publishing
- * Why this perspective fits the persona
- * What insight the post provides

3. Sources

Return:

```
```json
"sources": []
```
```

or valid source URLs.

Prefer source URLs already supplied by Member 2.

Do not invent URLs.

OPENAI + GEMINI

Support:

```
```env
OPENAI_API_KEY=
OPENAI_MODEL=
```
```

and:

```
```env
GEMINI_API_KEY=
GEMINI_MODEL=
```
```

Implement a clean provider strategy.

Preferred behavior:

```
```text
OPENAI_API_KEY available
↓
Use OpenAI
↓
```

If OpenAI fails



Try Gemini if configured

```

If only Gemini is configured:

```text

Use Gemini

```

If only OpenAI is configured:

```text

Use OpenAI

```

If neither is configured:

```text

Raise a clear configuration error

```

Do not expose API keys in logs.

DATABASE

Use:

```text

SQLite

SQLAlchemy 2.x

aiosqlite

```

Database configuration must come from:

```env

DATABASE\_URL=sqlite+aiosqlite:///./data/aura.db

```

Create SQLAlchemy model:

```text

Posts

```

Required fields:

```
```text
id
title
text
rationale
sources
published_time
created_at
```
```

Use:

- * SQLAlchemy 2.x typed mappings
- * Mapped[]
- * mapped_column()
- * async sessions
- * proper transaction handling
- * rollback on errors

Store sources safely, for example as JSON text in SQLite.

FEED API

Create:

```
```text
app/routes/feed.py
```
```

Endpoint:

```
```http
GET /api/agent/feed
```
```

It must return all published posts.

Response should be structured using Pydantic.

Example:

```
```json
{
 "total": 2,
 "posts": [
 {
 "id": 2,
 "title": "The Future of AI Agents",

```

```
 "text": "AI agents are becoming...",
 "rationale": "This topic...",
 "sources": [
 "https://example.com/source"
],
 "published_time": "2026-08-08T14:30:00Z",
 "created_at": "2026-08-08T14:30:00Z"
}
]
}
```

Return newest posts first.

## MANUAL PUBLISH API

Create:

```
```text
app/routes/publish.py
```
```

Endpoint:

```
```http
POST /api/publish
```
```

This endpoint is for:

- \* Hackathon demonstration
- \* Testing
- \* Manual triggering

It must use the SAME production publishing pipeline as the scheduler.

Do not duplicate publishing logic.

The endpoint should:

```
```text
POST /api/publish
↓
Member 2 approved topic
↓
Member 1 persona
↓
AI generation
↓
```

SQLite

...

It should not accept arbitrary topics directly.

DASHBOARD API

Create:

```
```text
app/routes/dashboard.py
```
```

Endpoint:

```
```http
GET /api/dashboard
```
```

Return:

```
```text
Total Posts
Scheduler Status
Latest Post
Last Run Time
```
```

Prefer this structure:

```
```json
{
 "total_posts": 5,
 "scheduler_status": "running",
 "latest_post": {
 "id": 5,
 "title": "AI Agents Are Becoming Infrastructure",
 "text": "...",
 "published_time": "..."
 },
 "last_run_time": "...",
 "last_run_success": true,
 "last_run_error": null
}
```
```

Track scheduler execution status safely.

HEALTH CHECK

Create:

```
```http
GET /
```
```

Return:

```
```json
{
 "status": "healthy",
 "service": "AURA Autonomous Publishing Module",
 "version": "1.0.0"
}
```
```

You may also create:

```
```http
GET /health
```
```

for a more detailed health response.

LOGGING

Create:

```
```text
app/utils/logger.py
```
```

Implement centralized logging.

Log:

- * Application startup
- * Application shutdown
- * Scheduler startup
- * Scheduler shutdown
- * Scheduler execution
- * Topic retrieval
- * Persona retrieval
- * AI provider selection

- * AI failures
- * Database failures
- * Successful publishing
- * API errors

Support:

```
```env
LOG_LEVEL=INFO
LOG_FILE=logs/aura.log
```
```

Never log:

- * API keys
- * Secrets
- * Authorization headers
- * Sensitive persona information unnecessarily

Use structured, readable log messages.

ERROR HANDLING

Use try/except appropriately.

Handle:

Member 2 failures

- * Timeout
- * Connection error
- * HTTP errors
- * Invalid JSON

Member 1 failures

- * Timeout
- * Connection error
- * HTTP errors
- * Invalid JSON

AI failures

- * Provider unavailable
- * Invalid response
- * Empty response

* Invalid JSON

Database failures

- * Commit failure
- * Connection failure
- * Rollback

Return appropriate HTTP status codes.

Do not expose internal stack traces to API clients.

Log full exceptions internally.

HTTP CLIENT

Use:

```
```text
httpx.AsyncClient
```
```

Do not use blocking requests.

Configure timeout through:

```
```env
REQUEST_TIMEOUT_SECONDS=15
```
```

Always close HTTP clients properly.

PROJECT STRUCTURE

Generate EXACTLY this structure:

```
```text
aura-publisher/
■
■■■ app/
■ ■■■ __init__.py
■ ■■■ main.py
■ ■■■ database.py
■ ■■■ models.py
■ ■■■ config.py
■ ■■■ scheduler.py
```
```

```

■ ■■■ generator.py
■ ■
■ ■■■ routes/
■ ■ ■■■ __init__.py
■ ■ ■■■ feed.py
■ ■ ■■■ dashboard.py
■ ■ ■■■ publish.py
■ ■
■ ■■■ services/
■ ■ ■■■ __init__.py
■ ■ ■■■ publisher.py
■ ■
■ ■■■ utils/
■ ■■■ __init__.py
■ ■■■ logger.py
■
■■■ data/
■ ■■■ .gitkeep
■
■■■ logs/
■ ■■■ .gitkeep
■
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ Dockerfile
■■■ README.md
...

---
```

IMPORTANT ARCHITECTURAL RULE

The central publishing workflow must live in:

```

```text
app/services/publisher.py
...

```

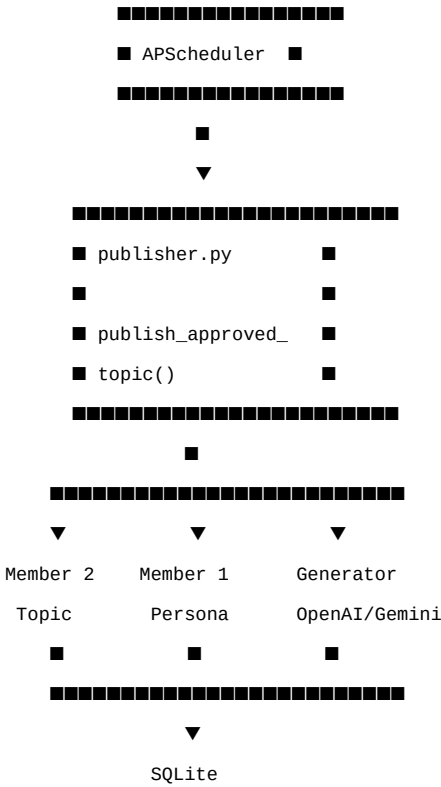
The scheduler must call the service.

The manual `/api/publish` endpoint must call the SAME service.

Do NOT duplicate the workflow.

Architecture:

```text



```

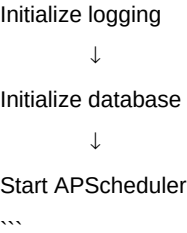
---

# FASTAPI LIFESPAN

Use FastAPI's modern lifespan mechanism.

On startup:

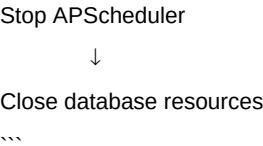
```text



```

On shutdown:

```text



```

Do not use deprecated startup/shutdown patterns if a modern lifespan implementation is appropriate.

---

# DEPENDENCY INJECTION

Use:

```
```python
Depends(get_db)
```
```

for database sessions.

Do not create global database sessions inside routes.

---

# PYDANTIC

Use Pydantic v2 models for:

- \* API responses
- \* Configuration
- \* Validation where appropriate

Use proper response\_model declarations.

---

# DOCKER

Create a production-oriented Dockerfile.

Requirements:

- \* Python 3.12+
- \* Slim base image
- \* Install requirements
- \* Copy application
- \* Create `/app/data`
- \* Create `/app/logs`
- \* Expose port 8000
- \* Run Uvicorn

Command:

```
```bash
uvicorn app.main:app --host 0.0.0.0 --port 8000
```
```

---

# ENVIRONMENT VARIABLES

Create ``.env.example`` containing:

```
``env
APP_NAME=AURA Autonomous Publishing Module
APP_VERSION=1.0.0
ENVIRONMENT=development

DATABASE_URL=sqlite+aiosqlite:///./data/aura.db

OPENAI_API_KEY=
OPENAI_MODEL=gpt-4o-mini

GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.5-flash

MEMBER1_PERSONA_URL=http://localhost:8002/api/persona
MEMBER2_TOPICS_URL=http://localhost:8001/api/topics/approved

SCHEDULER_ENABLED=true
SCHEDULER_INTERVAL_MINUTES=30

REQUEST_TIMEOUT_SECONDS=15

LOG_LEVEL=INFO
LOG_FILE=logs/aura.log
``
```

Never commit the real ``.env``.

---

## README REQUIREMENTS

The README must explain:

1. Project overview
2. Architecture
3. Member responsibilities
4. Installation
5. Virtual environment
6. Dependencies
7. Environment variables
8. Running locally
9. Running with Docker
10. API endpoints
11. Scheduler
12. Database
13. AI provider configuration
14. Member 1 integration

- 15. Member 2 integration
- 16. Example API responses
- 17. Testing the manual publish endpoint
- 18. Production deployment considerations

Include commands for Windows and Linux/macOS where useful.

---

## PRODUCTION QUALITY REQUIREMENTS

The generated project must:

- \* Use async I/O
- \* Use type hints
- \* Use clean architecture
- \* Avoid duplicated code
- \* Use dependency injection
- \* Validate external responses
- \* Handle exceptions
- \* Log important events
- \* Avoid leaking secrets
- \* Use environment configuration
- \* Have clean service boundaries
- \* Be Docker-ready
- \* Be easy for three developers to integrate
- \* Be understandable during a hackathon demo

Do not over-engineer the project with unnecessary microservices.

---

## OUTPUT FORMAT — VERY IMPORTANT

Generate the project **\*\*ONE FILE AT A TIME\*\***.

Follow this EXACT order:

### 1. Folder Structure

Show the complete tree.

### 2. requirements.txt

Show the complete file.

### 3. app/config.py

Show the complete file.

#### **4. app/database.py**

Show the complete file.

#### **5. app/models.py**

Show the complete file.

#### **6. app/generator.py**

Show the complete file.

#### **7. app/scheduler.py**

Show the complete file.

#### **8. app/routes/feed.py**

Show the complete file.

#### **9. app/routes/dashboard.py**

Show the complete file.

#### **10. app/routes/publish.py**

Show the complete file.

#### **11. app/services/publisher.py**

Show the complete file.

#### **12. app/utils/logger.py**

Show the complete file.

#### **13. app/\*\*init\*\*.py**

Show the complete file.

#### **14. app/routes/\*\*init\*\*.py**

Show the complete file.

## 15. app/services/\*\*init\*\*.py

Show the complete file.

## 16. app/utils/\*\*init\*\*.py

Show the complete file.

## 17. app/main.py

Show the complete file.

## 18. .env.example

Show the complete file.

## 19. .gitignore

Show the complete file.

## 20. Dockerfile

Show the complete file.

## 21. README.md

Show the complete file.

---

# CRITICAL OUTPUT RULES

DO NOT:

- \* Skip files
- \* Give pseudocode
- \* Give incomplete snippets
- \* Use `...`
- \* Say "implement this yourself"
- \* Leave TODOs for core functionality
- \* Invent Member 1 implementation
- \* Invent Member 2 implementation
- \* Add topic discovery
- \* Add memory



- \* Add web scraping
- \* Add unnecessary frontend code
- \* Add unnecessary authentication unless explicitly required

Every file must contain complete executable code.

Make imports consistent across all files.

Ensure the project can actually start with:

```
```bash
unicorn app.main:app --reload
```
```

Ensure the Docker container can start with:

```
```bash
docker build -t aura-publisher .
docker run --env-file .env -p 8000:8000 aura-publisher
```
```

---

## FINAL SECTION

After all files have been generated, provide:

### How to Run

Give exact commands.

### How to Configure

Explain ``.env`.

### How Member 1 Integrates

Show the expected:

```
```http
GET /api/persona
```
```

request and response.

### How Member 2 Integrates

Show the expected:

```
```http
GET /api/topics/approved
```
```

request and response.

## End-to-End Demo

Explain exactly how I can demonstrate:

```
```text
POST /api/publish
  ↓
Approved topic
  ↓
Persona
  ↓
AI generation
  ↓
SQLite
  ↓
GET /api/agent/feed
  ↓
GET /api/dashboard
```
```

## Hackathon Demo Checklist

Provide a short checklist covering:

- \* Server running
- \* Database initialized
- \* AI API configured
- \* Member 1 API connected
- \* Member 2 API connected
- \* Manual publishing tested
- \* Feed tested
- \* Dashboard tested
- \* Scheduler verified
- \* Docker tested

Remember:

**\*\*I am Member 3. Keep my implementation strictly limited to the Autonomous Publishing Module.\*\***

The final result must be a **\*\*complete, runnable, modular, production-oriented FastAPI backend\*\***, not a tutorial or conceptual example.

PROMPT—2

What You Can Build

## 1. FastAPI Backend ■

Create the backend server.

Example APIs:

GET /

GET /api/agent/feed

POST /api/publish

GET /api/dashboard

GitHub Commit

Setup FastAPI backend

## 2. Scheduler ■■■■■

This is your biggest responsibility.

Use APScheduler.

Flow:

Every 30 Minutes

↓

Check Approved Topic

↓

Generate Post

↓

Save

↓

Sleep

Commit

Implement autonomous scheduler

## 3. AI Content Generator ■■■■

Integrate

Gemini

OpenAI

Function

generate\_post(topic, persona)

Commit

Integrate Gemini API

4. Feed API ■■■■

GET /api/agent/feed

Returns

```
{
 "posts": []
}
```

Commit

Create feed API

5. Dashboard ■■■■

Build a simple dashboard.

Show

Latest Posts

Scheduler Status

Last Published Time

Total Posts

Commit

Create dashboard

6. Database ■■■■

Store

Post

Title

Text

Sources

Created Time

Commit

Setup database

7. Logging ■■

Store logs

10:30

Scheduler Started

10:31

Post Published

Commit

Add logging

8. Docker ■■■

Create

Dockerfile

Commit

Dockerize publisher

9. README ■■

Explain

Installation

APIs

Running Project

Commit

Update documentation

Extra Features (These Impress Judges)

Publish Queue

Instead of publishing immediately

Approved Topic

↓

Queue

↓

Scheduler

↓

Publish

Retry System

If AI API fails

Try Again

↓

3 Attempts

↓

Log Error

Health Check

GET /health

Response

```
{
 "status": "running"
}
```

Statistics API

GET /api/stats

Return

```
{
 "published": 12,
 "pending": 3,
 "failed": 1
}
```

Dashboard Cards

Posts Today

Scheduler Running

Next Publish

Last Publish

Total Posts

Your GitHub Contribution

If you complete these, your commits will naturally look like:

Initialize publisher module

Setup FastAPI

Configure SQLite

Create Post model

Add Pydantic schemas

Integrate Gemini API

Implement scheduler

Create publish service

Create feed API

Create dashboard

Add logging

Implement retry mechanism

Dockerize project

Update README

Refactor code

That's 15+ meaningful commits.

PROMPT—3

1) First connect to the repo

If you are invited, accept the GitHub invitation first. Then clone the repo:

```
git clone https://github.com/asthasingh015/AI-Project.git
```

```
cd AI-Project
```

2) Create your branch

```
git checkout -b publisher
```

```
git push -u origin publisher
```

3) Build in small steps and commit after each feature

Commit 1

Create the basic folder structure.

Example:

```
mkdir publisher
```

```
mkdir publisher/app publisher/routes publisher/services publisher/utils
```

```
touch publisher/main.py
```

Commit message:

```
git add .
```

```
git commit -m "Initialize publisher module structure"
```

```
git push
```

Commit 2

Add requirements.txt and dependencies.

Commit message:

```
git commit -m "Add project dependencies"
```

Commit 3

Set up FastAPI app in main.py.

Commit message:

```
git commit -m "Setup FastAPI application"
```

Commit 4

Add config file and .env.example.

Commit message:

```
git commit -m "Add configuration and environment example"
```

Commit 5

Create SQLite database connection.

Commit message:

```
git commit -m "Setup SQLite database"
```

Commit 6

Create Post model.

Commit message:

```
git commit -m "Create post model"
```

Commit 7

Add Pydantic schemas.

Commit message:

```
git commit -m "Add API schemas"
```

Commit 8

Build AI generator function.

Commit message:

```
git commit -m "Implement AI content generator"
```

Commit 9

Add scheduler with APScheduler.

Commit message:

```
git commit -m "Implement autonomous scheduler"
```

Commit 10

Create publish service.

Commit message:

```
git commit -m "Create publishing service"
```

Commit 11



Add feed API.

Commit message:

```
git commit -m "Create feed API"
```

Commit 12

Add manual publish API for testing.

Commit message:

```
git commit -m "Add manual publish endpoint"
```

Commit 13

Add dashboard API.

Commit message:

```
git commit -m "Create dashboard API"
```

Commit 14

Add logging and error handling.

Commit message:

```
git commit -m "Add logging and exception handling"
```

Commit 15

Update README and final documentation.

Commit message:

```
git commit -m "Update documentation and final cleanup"
```

4) Push after every commit

```
git push origin publisher
```

5) Final result

Your publisher branch will have a clean history like this:

Initialize publisher module structure

Add project dependencies

Setup FastAPI application

Add configuration and environment example

Setup SQLite database

Create post model

Add API schemas

Implement AI content generator

Implement autonomous scheduler

Create publishing service

Create feed API

Add manual publish endpoint

Create dashboard API

Add logging and exception handling

Update documentation and final cleanup

6) Best practice